

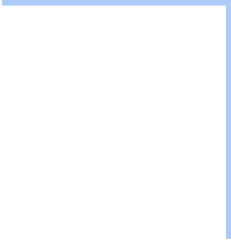


TOPIC-FIVE

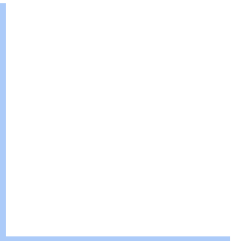
Kubernetes Administration

Author: [Arash Foroughi](#)





Introduction to Kubernetes



Introduction to Kubernetes

- **Kubernetes** is an open-source **container-orchestration** platform used for maintaining & deploying a group of containers.
- International Certification: **Certified Kubernetes Administration (CKA)**
- First time released at **2014** by **Google Inc.**
- It maintained by **Cloud Native Computing Foundation (CNCF)** in **2016**.
- Kubernetes has written in **Go/Golang** programming language.
- It has become standard for containerized apps in cloud, with the main Cloud Providers (**AWS, Azure, GCP, IBM & Oracle**).



Introduction to Kubernetes

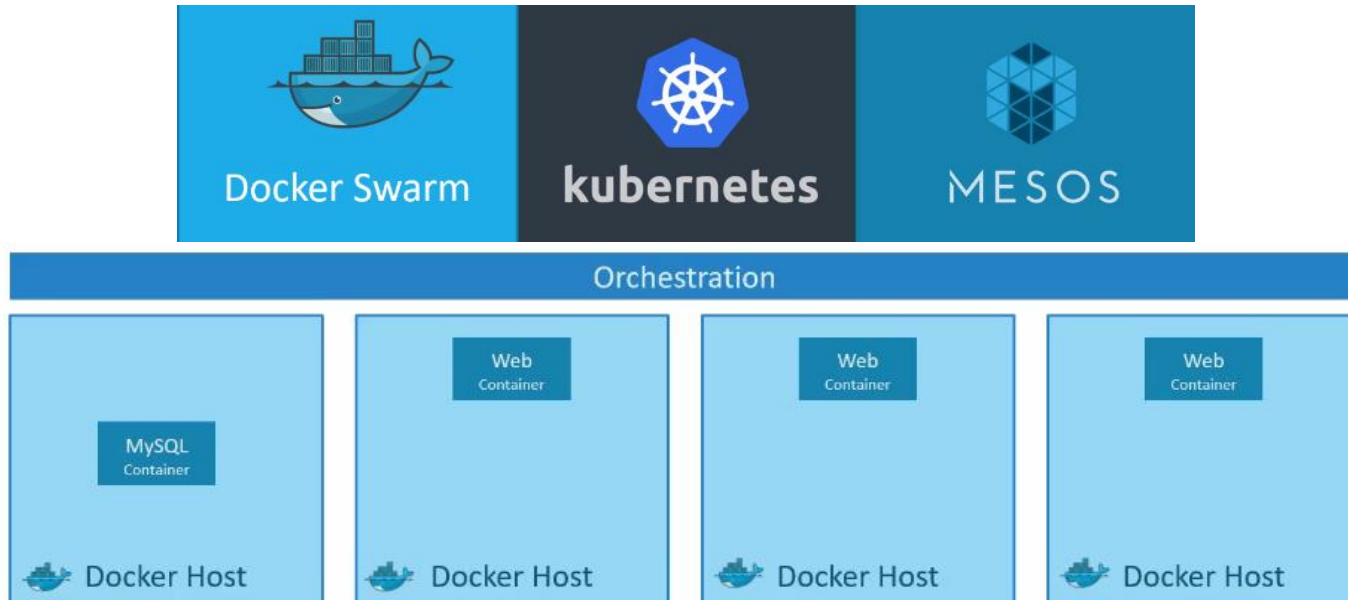
- The name "**Kubernetes**" stems from an **ancient Greek word** for "**helmsman**", (someone who steers a ship, like a container ship) which explains **ship wheel** logo.
- The **ship wheel** has **seven** sides, in homage to the original name for the project.

KUBERNETES
K + (UBERNETE) + S
8
K8s

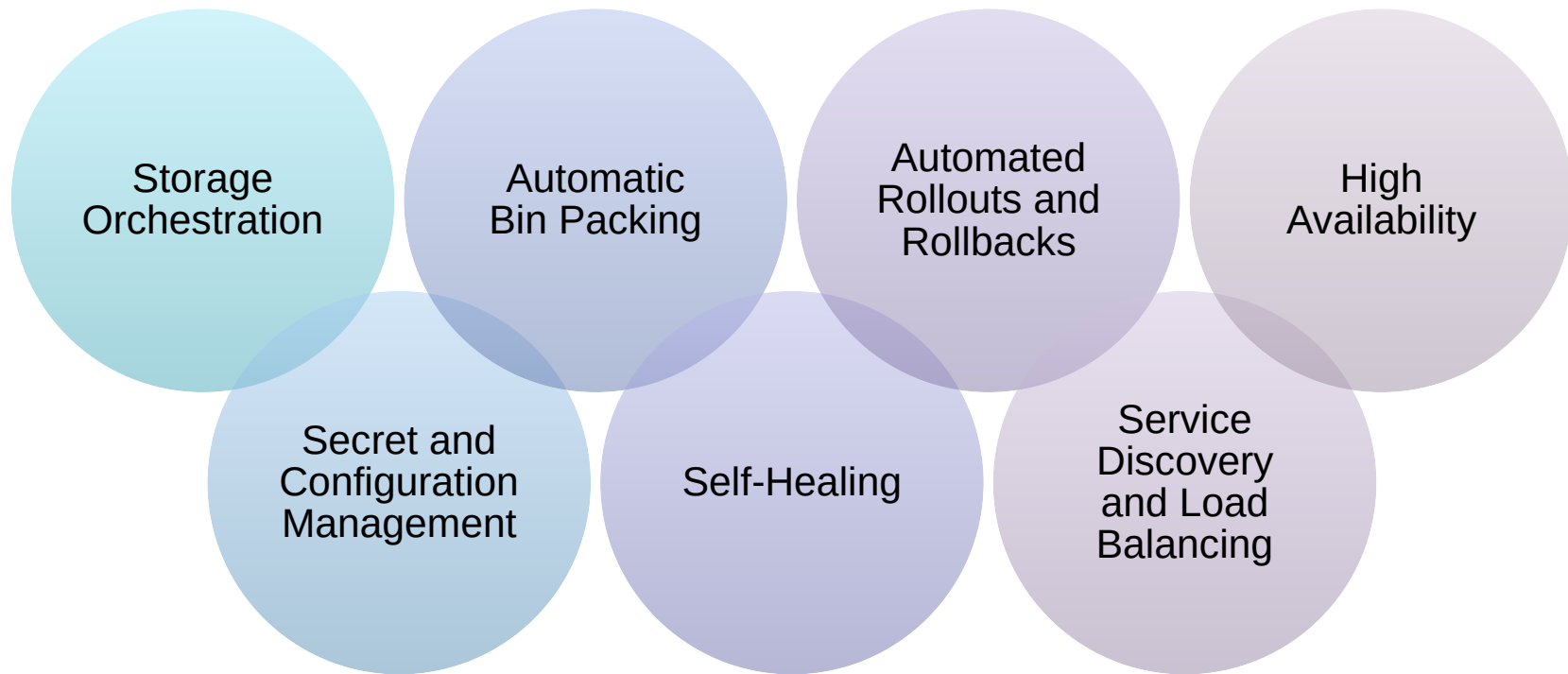


Container Orchestration

- **Container orchestration** is the **automatic process** of managing or scheduling the work of **individual containers** for applications based on **microservices** within multiple clusters.

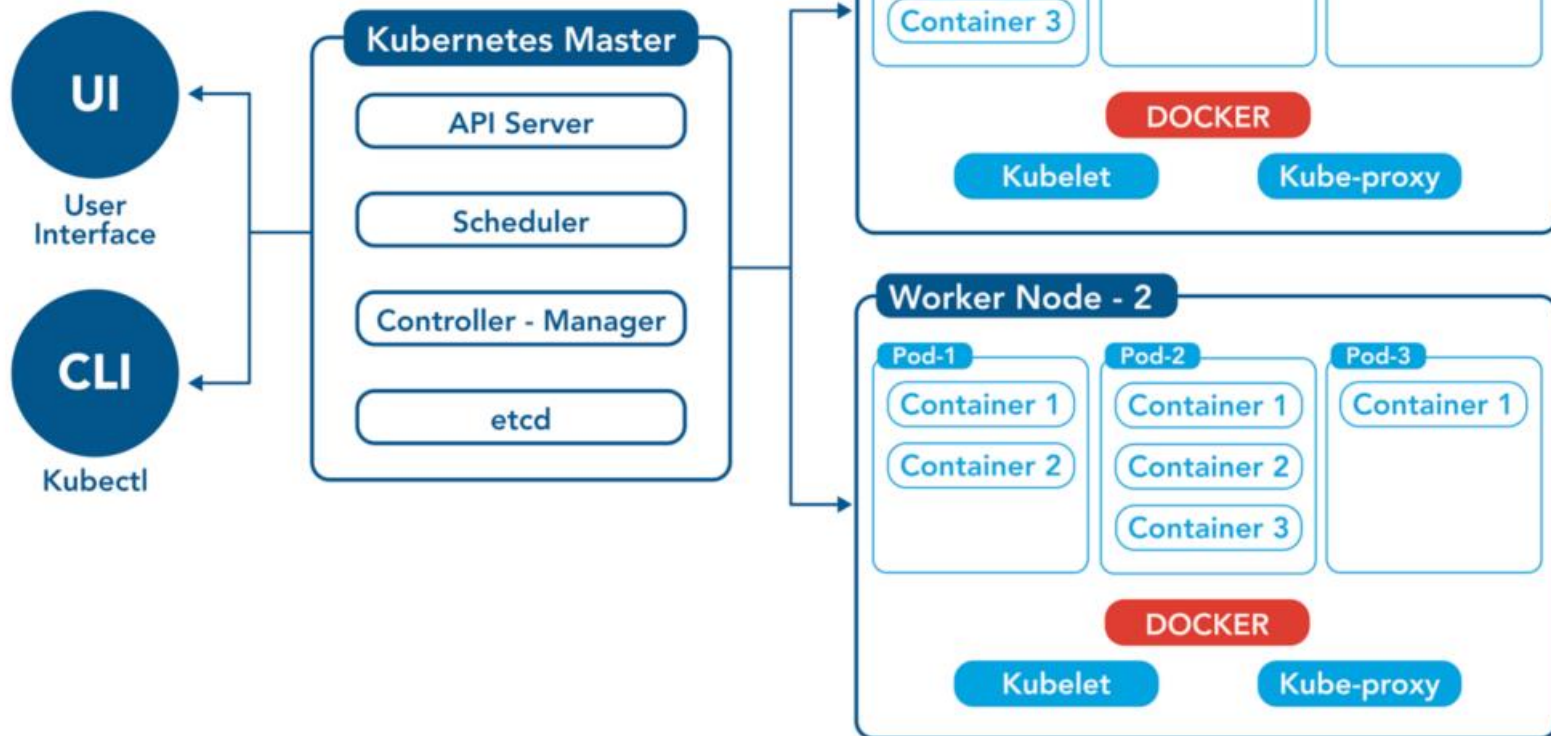


Kubernetes Functionalities



Kubernetes Architecture

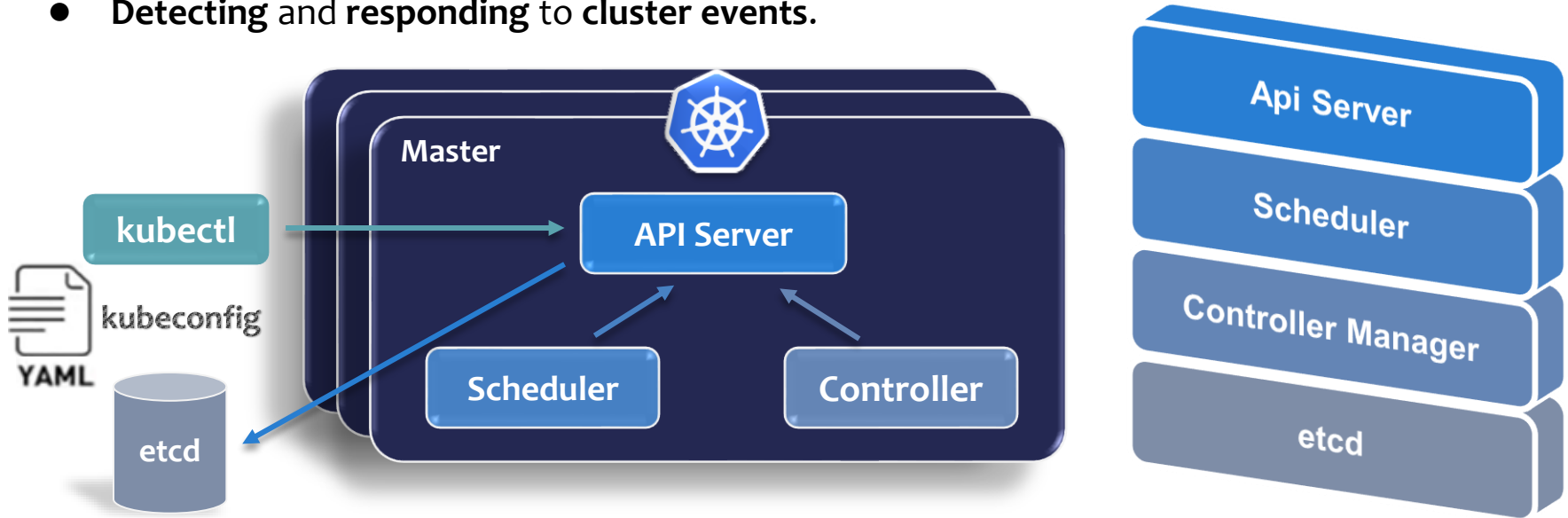
Kubernetes Architecture



Control Plane Components

Control Plane Components

- The **main machine** that controls the nodes.
- **Managing & controlling & orchestrating** of the worker nodes.
- Make **global decisions** about **cluster**, like scheduling.
- **Detecting** and **responding** to **cluster events**.

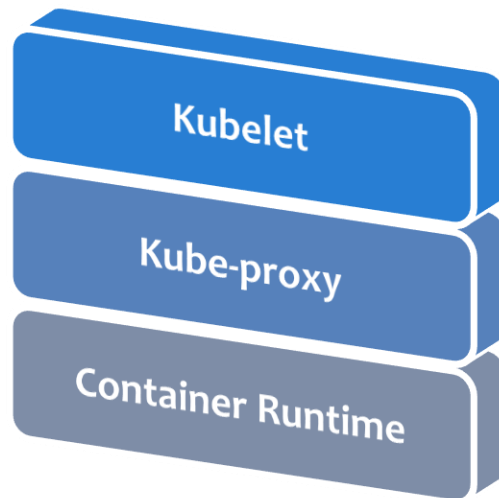
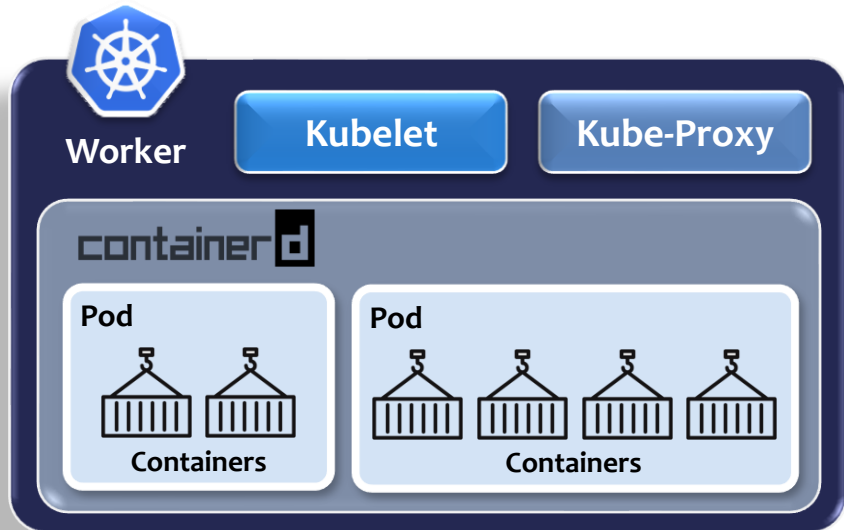


Service Name	Slogan	Description
API Server	Exposes the API	- Everything in kubernetes cluster will connect and talk to this API Server. - It is the front-end tools for sending Kubeconfig through kubectl CLI. - API server validates & configures required data for worker nodes. - Communicate with other libraries & components using RESTful interface.
Etcd or Cluster Store	Key-Value store	- It stands for “/etc distributed”. - A place for storing config files of clusters there. (Accessible only by API Server) - It is a key-value based storage for manage critical information. - Highly recommended to have a backup plan for etcd in a cluster.
Scheduler	Assign work to nodes	- Decide where to start the workload, on which node should it start the application. - Choosing a node for newly created pods on kubernetes. - individual & collective resource requirements, hardware/software/policy constraints, affinity & anti-affinity specifications, data locality & inter-workload interference.
Controller Manager	Watch for changes	- There are some controllers here that each controller is a separate process. - They are all compiled into a single binary and run in a single process. - It includes: Node/Replication/Endpoints/Service-Account/Token Controller.

Worker Node Components

Worker Node Components

- **Nodes**, **Worker nodes** or **minions**.
- It can be **physical** or a **virtual** machine based on cluster type.
- **Each node** containing **necessary services** for running **pods**.
- **Pod** is the **smallest element** in kubernetes which maintain **set of containers**.



Service Name	Slogan	Description
Container Runtime	Running Containers on the Worker	- The software which is responsible for running containers on nodes. - Such as Docker, containerd, CRI-O, rkt, Podman, LXC, ... - It will pull containers, start/stop/remove and everything can done with Docker.
Kubelet	Main Controller Manager Agent	- The agent that runs on each node & makes sure containers are running in a pod. - Compare running containers with Received PodSpecs. - Reassure from status of running containers & add/remove containers in pods.
Kube-Proxy	Kubernetes Networking	- It is a proxy service that runs on each node & helps in making services available to the external host. It helps to forwarding requests to assigned networks. - Responsible for configuring network rules for each nodes. - Assigning One Single IP to all containers in a pod.

Pods Theory

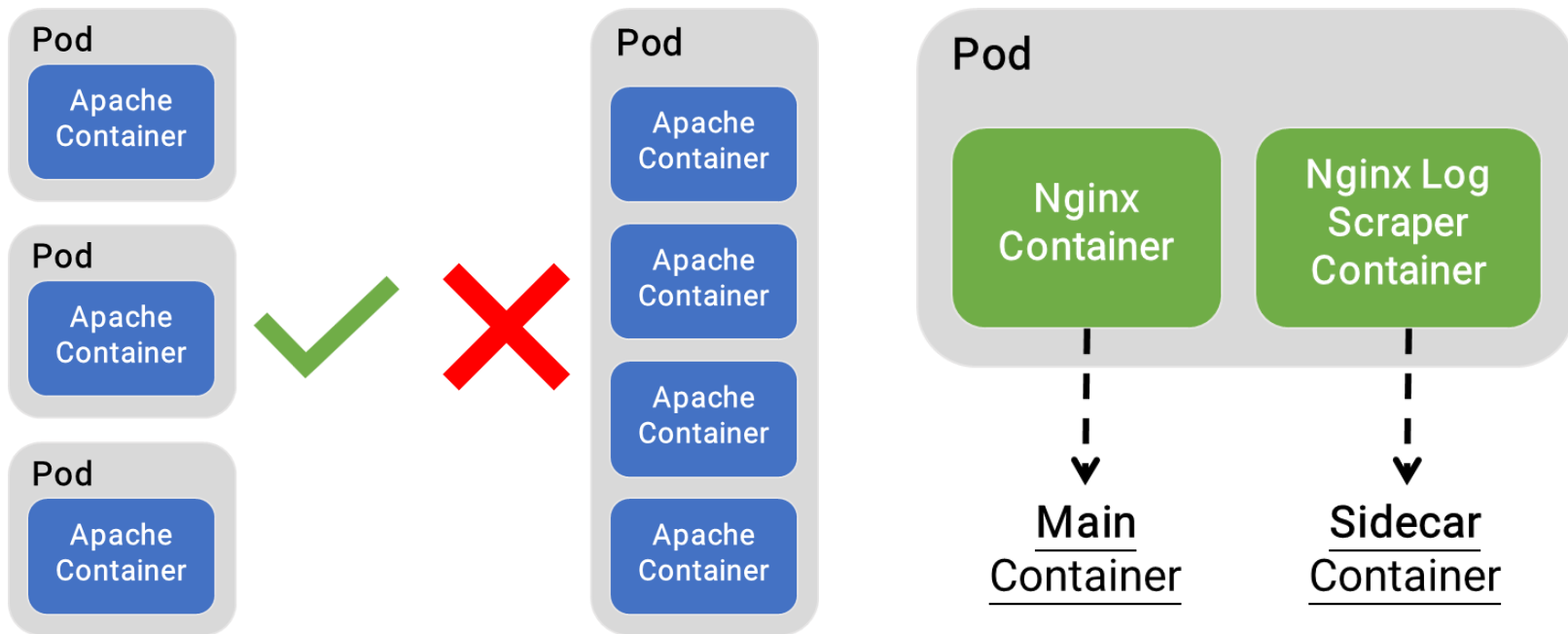
Pods Theory

- Pods are the **smallest**, most basic **deployable objects** in Kubernetes.
- A Pod represents a **single instance** of a **running process** in your cluster.
- Pods contain **one or more containers**, such as **Docker** containers.
- When a Pod runs **multiple containers**, containers are managed as a **single entity** & share Pod's **IP & resources**.
- Kubernetes manage **Pods, not containers!**



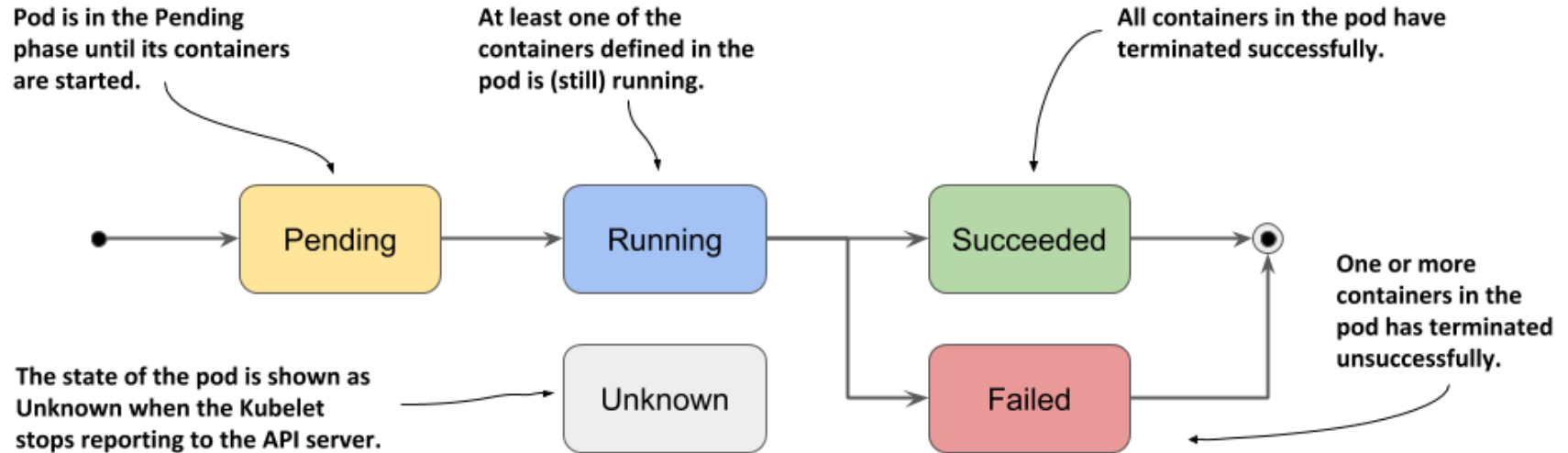
Scaling in Pods

- **Pod Scaling** in Kubernetes will be done in **Horizontally**, like below:



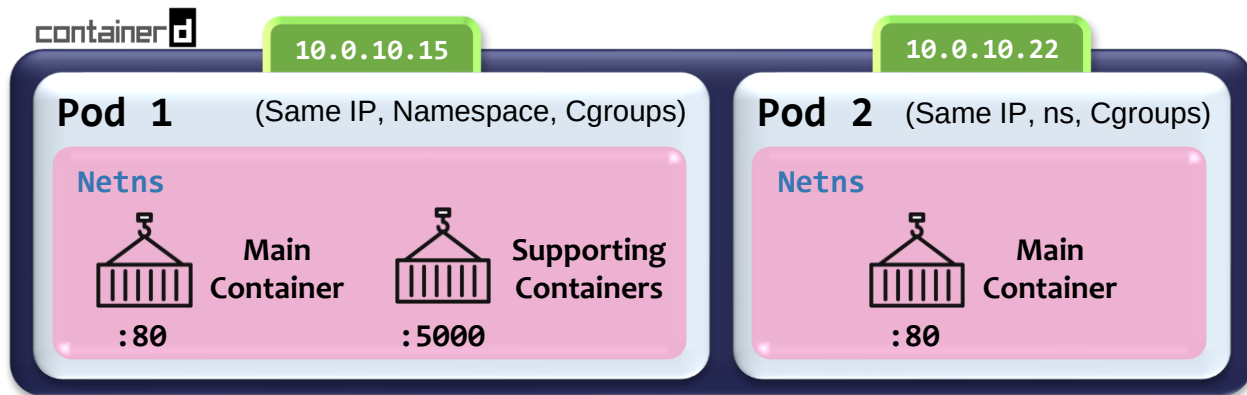
Pod Life Cycle

- Pods have a lifecycle, starting in **Pending** phase, moving through **Running** if at least has one primary container, then through either the **Succeeded** or **Failed** phases.



Pods Characteristics

- Each **Pod** whether it has how many containers, it just has **one IP Address**.
- Each **IP Address** is a member of a one **Network Namespace**.
- Each pod has its own **cgroups**.
 - **Cgroups** are the **kernel feature** that allows to **set limits for CPU, memory, & disk I/O** for one or more processes. By using cgroups, a **process & its network** can be **isolated**.
- But at last each container under a Pod, can have **different port**.



K8s Different Versions

K8s Different Versions



minikube





Minikube Installation

Install Dependencies

```
[node1 ~]# apt install wget conntrack apt-transport-https virtualbox virtualbox-ext-pack
```

Install Minikube

```
[node1 ~]# wget https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
[node1 ~]# cp minikube-linux-amd64 /usr/local/bin/minikube
[node1 ~]# chmod 755 /usr/local/bin/minikube; minikube version
```

Install Kubectl

```
[node1 ~]# curl -LO https://storage.googleapis.com/kubernetes-release/release/`curl -s
https://storage.googleapis.com/kubernetes-
release/release/stable.txt`/bin/linux/amd64/kubectl
[node1 ~]# chmod +x ./kubectl; mv ./kubectl /usr/local/bin/kubectl; kubectl version -o json
```

Start Minikube

```
[node1 ~]# minikube start --driver=none
[node1 ~]# kubectl config view
[node1 ~]# kubectl cluster-info
[node1 ~]# kubectl proxy --address='0.0.0.0' --disable-filter=true &
[node1 ~]# minikube dashboard --url
```

Minikube
Installation



Kubeadm Installation

===== Disable Swap =====

- Disable swap space & comment out swap entries in /etc/fstab :

```
[node1 ~]# swapoff -a; sed -i '\|swap|s|^|#|g' /etc/fstab
```

===== Install Containerd.io =====

- Install necessary packages:

```
[node1 ~]# apt update; apt install ca-certificates curl
```

- Create a directory for keyrings:

```
[node1 ~]# install -m 0755 -d /etc/apt/keyrings
```

- Download the Docker GPG key and save it to /etc/apt/keyrings/docker.asc:

```
[node1 ~]# curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o  
/etc/apt/keyrings/docker.asc
```

- Set read permissions for the Docker GPG key:

```
[node1 ~]# chmod a+r /etc/apt/keyrings/docker.asc
```

- Add the Docker repository to the package sources list:

```
[node1 ~]# echo "deb [arch=$(dpkg --print-architecture) signed-  
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.  
/etc/os-release && echo \"\$VERSION_CODENAME\") stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

- Install containerd.io package:

```
[node1 ~]# apt update; apt install containerd.io -y
```

===== Configure Containerd =====

- Retrieve the default containerd configuration and save it to /etc/containerd/config.toml:
`[node1 ~]# containerd config default > /etc/containerd/config.toml`
- Replace the value of "SystemdCgroup" from "false" to "true" in the config.toml file:
`[node1 ~]# sed -E -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml`
- Reload the systemd daemon to apply the changes:
`[node1 ~]# systemctl daemon-reload`
- Restart the containerd service to apply the updated configuration:
`[node1 ~]# systemctl restart containerd`

===== Install Kubeadm/Kubectl/Kubelet =====

- Load two recommended Kernel modules (br_netfilter & overlay):
[node1 ~]# modprobe br_netfilter overlay
- Enable IPv4 forwarding Kernel parameter in sysctl.conf:
[node1 ~]# sed -i 's/#net.ipv4.ip_forward=1/net.ipv4.ip_forward=1/g' /etc/sysctl.conf
- Reload sysctl settings & check if IPv4 forwarding is enabled:
[node1 ~]# sysctl --system; sysctl -a | grep net.ipv4.ip_forward
- Install necessary packages for Kubernetes:
[node1 ~]# apt update; apt install apt-transport-https ca-certificates curl conntrack -y
- Download and add the Kubernetes GPG key:
[node1 ~]# curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
- Add Kubernetes repository to sources.list:
[node1 ~]# echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
- Check the policy for kubeadm package for finding & installing the best version that is fit:
[node1 ~]# apt update; apt-cache policy kubeadm
[node1 ~]# apt install kubelet kubeadm kubectl -y
- Mark kubelet, kubeadm, and kubectl as held to prevent automatic upgrades:
[node1 ~]# apt-mark hold kubelet kubeadm kubectl

Kubernetes Initiation

===== Single Master Initiation =====

```
[node1 ~]# kubeadm init --apiserver-advertise-address 192.168.10.100
--pod-network-cidr 10.244.0.0/16
```

Your Kubernetes control-plane has initialized **successfully!**

➤ To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

➤ Alternatively, if you are the root user, you can run:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

Kubeconfig File

➤ You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

➤ Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 192.168.10.100:6443 --token w8u5t7.a95u24qgry6j --discovery-token-ca-cert-hash
```

Deploy a Pod Network (Flannel) to Cluster

- Platforms like Kubernetes assume that **each container (pod)** has a **unique** and **routable IP** inside the cluster. The advantage of this model is that it **removes** the **port mapping complexities** that come from sharing a **single host IP**.
- **Flannel** is responsible for **providing a layer 3 IPv4 network** between nodes of a cluster.

```
[node1 ~]# kubectl apply -f  
https://github.com/coreos/flannel/raw/master/Documentation/ku  
be-flannel.yml
```

```
[node1 ~]# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
Server-1	Ready	control-plane, master	25m	v1.20.1

```
[node1 ~]# kubectl get pods -A
```

Kubeadm Join Token Print

- For printing again the **kubeadm join command** with **SHA hash** certificate:

```
[node1 ~]# kubeadm token create --print-join-command
```

```
kubeadm join 192.168.0.7:6443 --token uzmsxh.nloe5k0ceb289du1 --discovery-  
token-ca-cert-hash sha256:d0ad667cbc6f316165f44239d399d34
```

```
[node1 ~]# kubeadm token --help
```

Available Commands:

create	Create bootstrap tokens on the server
delete	Delete bootstrap tokens on the server
generate	Generate a bootstrap token, but don't create it on server
list	List bootstrap tokens on the server

Kubectl Get Nodes

- For checking **list of nodes** which has been **added** to the cluster:

```
[node1 ~]# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
server-1	Ready	control-plane, master	25m	v1.29.1
server-2	Ready	<none>	24m	v1.29.1
server-3	Ready	<none>	24m	v1.29.1

```
[node1 ~]# kubectl describe nodes server-1
```

```
[node1 ~]# kubectl describe nodes server-2
```

```
[node1 ~]# kubectl label node node2 node-role.kubernetes.io/worker=worker
```

```
[node1 ~]# kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	29m
kube-node-lease	Active	29m
kube-public	Active	29m
kube-system	Active	29m

Running Workloads on Master Node

- By default, **no workloads** will run on the **master node**.
- If you want to **allow pods** to run **on the master node** as well. This is done by a process called "**tainting**" the host.
- On the master, **run below command** to allow workloads can run on master node as well.

```
[node1 ~]# kubectl taint node $HOSTNAME key:NoSchedule
node/server-1 tainted
```

Kubectl Bash Auto Completion

- `bash-completion` is provided by many package managers.

```
[node1 ~]# apt-get install bash-completion
```

- Now ensure `kubectl` completion script gets sourced in all your shell sessions:

```
[node1 ~]# kubectl completion bash | tee /etc/bash_completion.d/kubectl
```

```
[node1 ~]# kubeadm completion bash | tee /etc/bash_completion.d/kubeadm
```

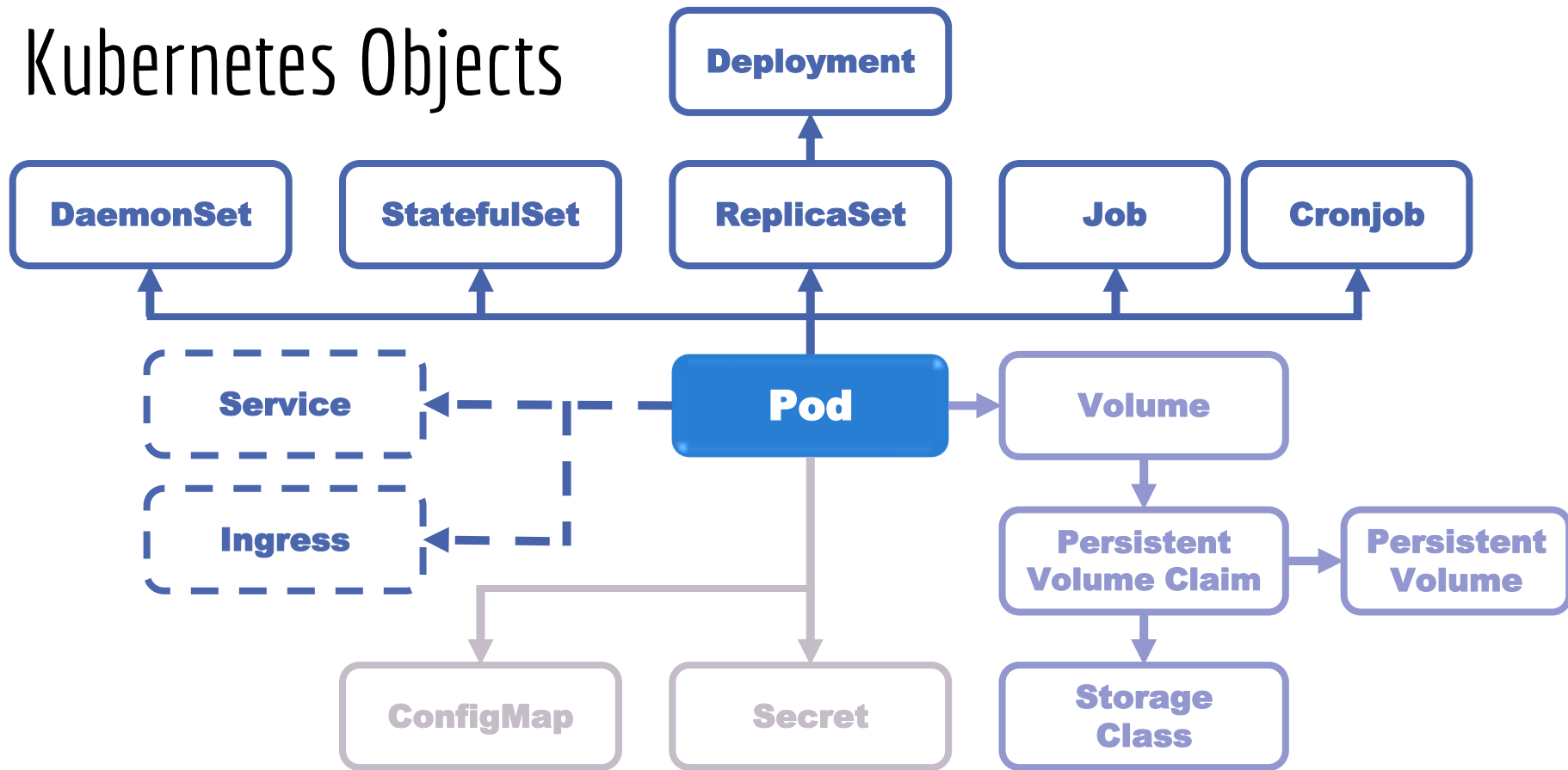
```
[node1 ~]# bash -l
```

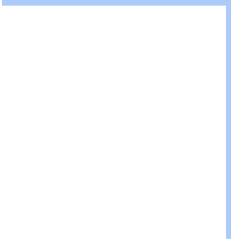
Kubernetes Objects

What are Kubernetes Objects?

- Kubernetes objects are **persistent entities** in the Kubernetes system.
- Kubernetes uses these **entities** to **represent** the **state of your cluster**.
- Specifically, they can describe:
 - *What containerized applications are running (and on which nodes).*
 - *The resources available to those applications.*
 - *The policies how those applications behave, such as restart policies, upgrades, and fault-tolerance.*
- **Manifest files** serve as the blueprint for defining the state of your cluster in YAML/JSON.

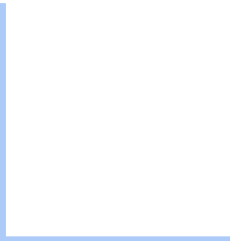
Kubernetes Objects





Kubernetes Objects

Namespaces

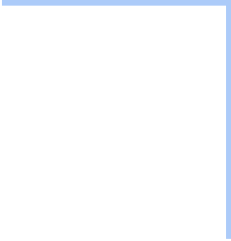


Namespaces in Kubernetes

- **Namespaces** provides a mechanism for **isolating groups** of resources within a single cluster.
- Namespaces are **intended** for use in **environments** with **many users** spread across multiple teams, or projects.

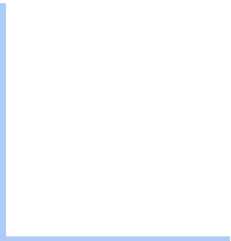
Namespaces	Description
default	The default namespace for objects with no other namespace
Kube-system	The namespace for objects created by the Kubernetes system
Kube-public	This namespace is created automatically & is readable by all users (including those not authenticated). This namespace is mostly reserved for cluster usage, in case that some resources should be visible & readable publicly throughout the whole cluster. The public aspect of this namespace is only a convention, not a requirement.
Kube-node-lease	This namespace holds <u>Lease</u> objects associated with each node. Node leases allow the Kubelet to send <u>heartbeats</u> so that the control plane can detect node failure.

```
[node1 ~]$ kubectl create namespace myns-1
```



Kubernetes Objects

Pods



Manifest File Fields

- **apiVersion** - Which version of the Kubernetes API you're using to create this object
- **kind** - What kind of object you want to create
- **metadata** - Data that helps uniquely identify the object, including a name string, UID, and optional namespace
- **spec** - What state you desire for the object

```
apiVersion: v1
kind: Namespace
metadata:
  name: ns-1
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: ns-1
  labels:
    app: myapp
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
      imagePullPolicy: Always
      ports:
        - containerPort: 80
      restartPolicy: Always
---
```

NS
Pod

Getting Status of Objects

```
[node1 ~]$ kubectl get pods -o wide -n ns-1
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-pod	1/1	Running	0	9s	10.5.2.3	node3

```
[node1 ~]$ kubectl get pods nginx-pod -o yaml -n ns-1
```

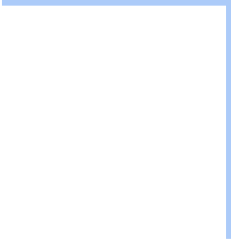
```
[node1 ~]$ kubectl describe pods nginx-pod -n ns-1
```

Output format	Description
<code>-o custom-columns=<spec></code>	Print a table using a comma separated list of custom columns
<code>-o custom-columns-file=<filename></code>	Print a table using the custom columns template in the <filename> file
<code>-o json</code>	Output a JSON formatted API object
<code>-o jsonpath=<template></code>	Print the fields defined in a jsonpath expression
<code>-o jsonpath-file=<filename></code>	Print the fields defined by the jsonpath expression in the <filename> file
<code>-o name</code>	Print only the resource name and nothing else
<code>-o wide</code>	Output in plain-text format with any additional info, and for pods, node name is included
<code>-o yaml</code>	Output a YAML formatted API object

Kubectl Output Verbosity & Debugging

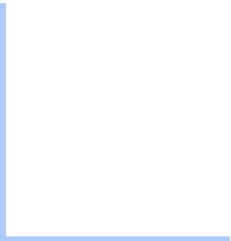
- Kubectl verbosity is controlled with the `-v` or `--v` flags followed by an integer representing log level.

Verbosity	Description
<code>--v=0</code>	Generally useful for this to always be visible to a cluster operator.
<code>--v=1</code>	A reasonable default log level if you don't want verbosity.
<code>--v=2</code>	Useful steady state information about service & important log messages. (recommended)
<code>--v=3</code>	Extended information about changes.
<code>--v=4</code>	Debug level verbosity.
<code>--v=5</code>	Trace level verbosity.
<code>--v=6</code>	Display requested resources.
<code>--v=7</code>	Display HTTP request headers.
<code>--v=8</code>	Display HTTP request contents.
<code>--v=9</code>	Display HTTP request contents without truncation of contents.



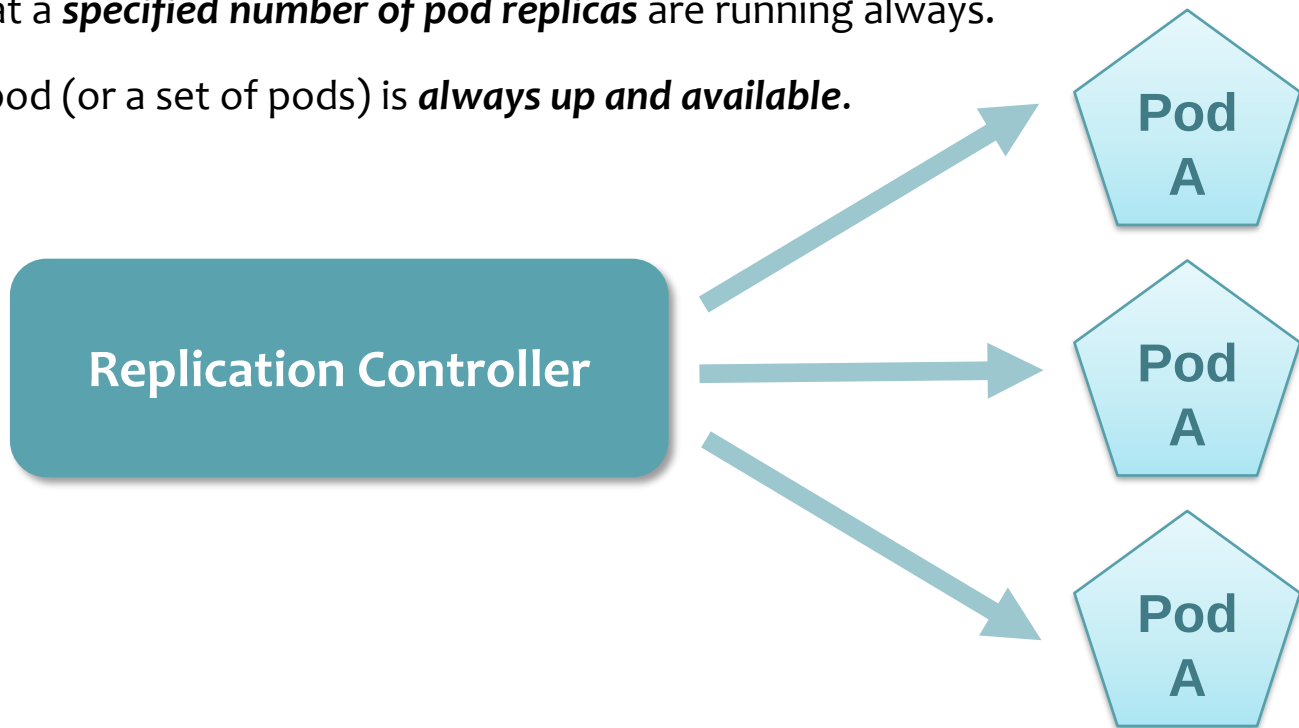
Kubernetes Objects

Replication Controller (RC)



ReplicationController (RC)

- An **RC** ensures that a *specified number of pod replicas* are running always.
- It ensures that a pod (or a set of pods) is *always up and available*.



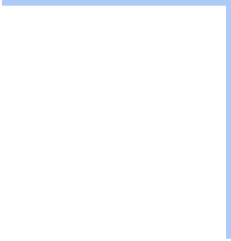
```
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
  namespace: default
spec:
  replicas: 5
  selector:
    app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

```
[node1 ~]$ kubectl apply -f pod.yaml
```

ReplicationController (RC)

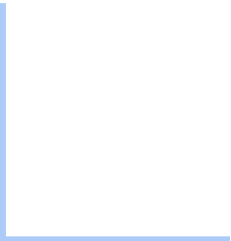
- You can scale *number of replicas* of an RC using `kubectl scale` command.
- Or by updating the replicas field in the **RC manifest Yaml file** and applying changes.

```
$ kubectl scale rc/nginx-rc --replicas=8
$ kubectl get rc -o wide
$ kubectl describe rc nginx-rc
$ kubectl logs -f rc/nginx-rc
$ kubectl exec -it nginx-pod -- /bin/bash
$ kubectl cp file1 nginx-rc:/app/data
```



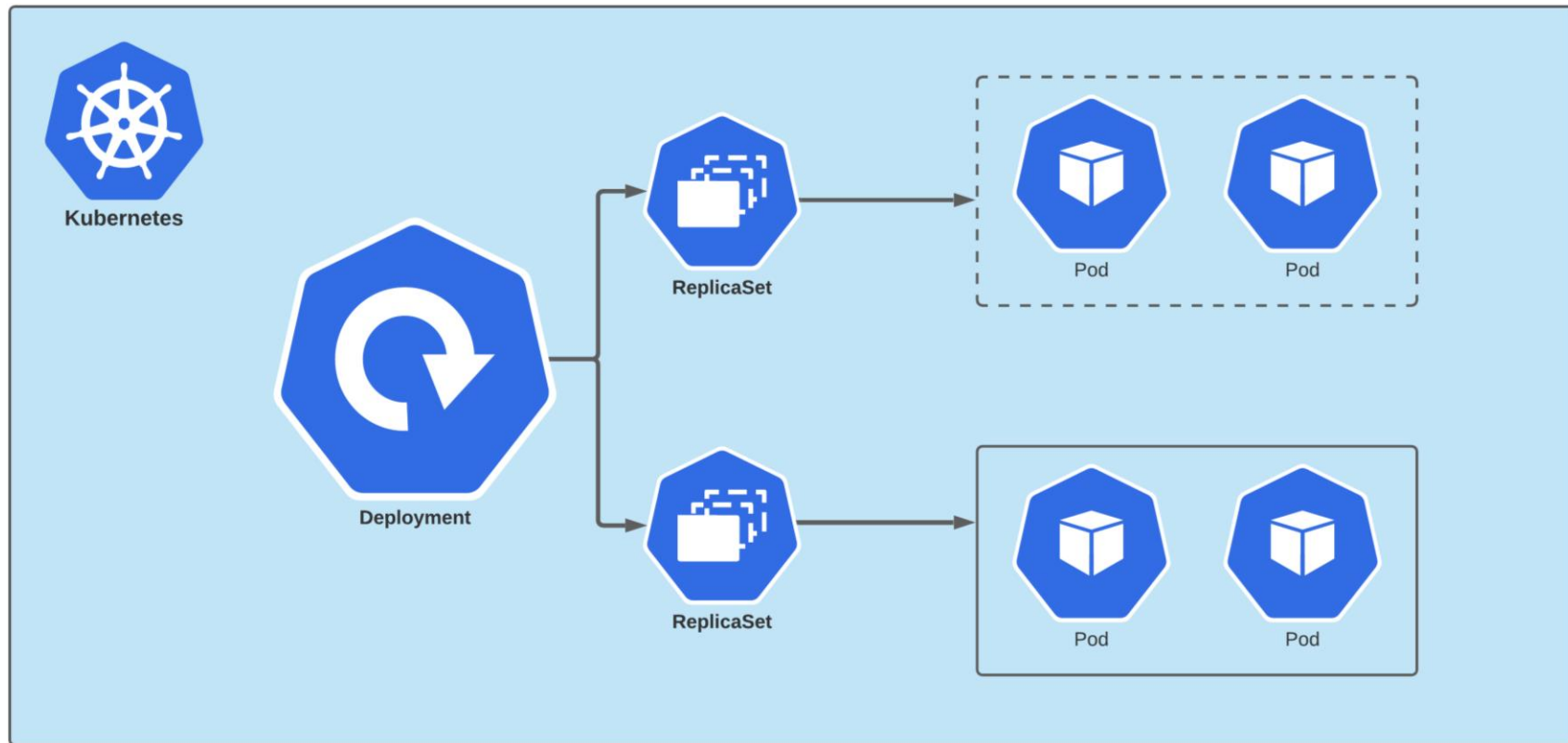
Kubernetes Objects

Deployments – ReplicaSets (RS)



Deployment

- **Deployment** is a higher-level of RC which it will control different versions of **ReplicaSets (RS)**.
- **Deployment** can handle **roll updates** and **rollbacks** to different **Revisions**.



Deployment

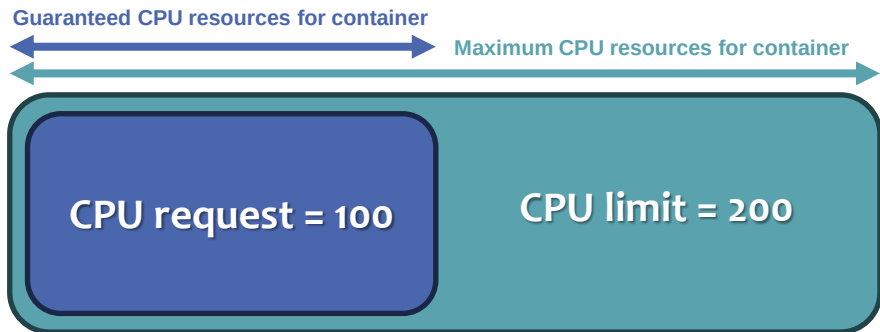
```
$ kubectl scale deploy/nginx-deploy --replicas=5
$ kubectl get deploy -o wide
$ kubectl get rs -o wide
$ kubectl get pods -o wide
$ kubectl describe deploy/nginx-deploy
$ kubectl describe rs/nginx-deploy-rs-name
$ kubectl logs -f deploy/nginx-deploy

-----
$ kubectl cordon server-2
$ kubectl uncordon server-2      Maintenance
$ kubectl drain server-2         Mode
$ kubectl cluster-info dump
$ kubectl cluster-info dump --output-directory /tmp
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  namespace: default
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Resource Requests & Limits

- **Resource requirements** and **limits** are used to specify the **amount of CPU** and **memory** resources that a container requires in order **to run** properly.



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          resources:
            requests:
              cpu: "100m"
              memory: "10M"
            limits:
              cpu: "200m"
              memory: "50M"
```

Revision Controlling with Deployment

- There are two ways for **Rolling Out** a new version of Deployment:

1. Through Command Line:

```
[node1 ~]$ kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1
```

```
[node1 ~]$ kubectl rollout status deployment/nginx-deployment
```

```
[node1 ~]$ kubectl rollout history deployment/nginx-deployment
```

```
[node1 ~]$ kubectl rollout pause deployment/nginx-deployment
```

```
[node1 ~]$ kubectl annotate deployment/nginx-deployment kubernetes.io/change-cause="image updated to 1.16.1"
```

```
[node1 ~]$ kubectl rollout undo deployment/nginx-deployment
```

```
[node1 ~]$ kubectl rollout undo deployment/nginx-deployment --to-revision=2
```

Rollout & Rollback

- **Rollout** is the process of updating a **Deployment** or **ReplicaSet** to a **new version** of an app.
- **Rollback** is performed by **updating container** with the **previous version** of the container image.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.17
        ports:
        - containerPort: 80
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.18
        ports:
        - containerPort: 80
```

Deployment Strategy Models

Rolling Updates

- are performed by gradually replacing instances of an old version of a container with instances of a new version. (default deployment strategy).

Recreate

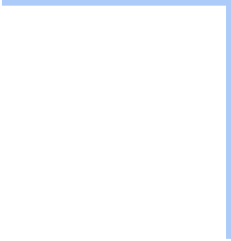
- strategy deletes all the old Pods before creating new ones. This can result in some downtime for your application.

Blue/Green

- strategy creates a new set of Pods running the updated version of your application alongside the old set of Pods running the previous version.

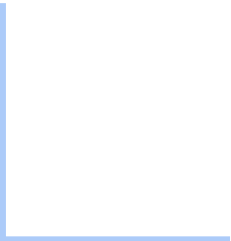
Canary

- strategy updates a small percentage of Pods with the new version of your application, while the rest of the Pods continue to run the previous version.



Kubernetes Objects

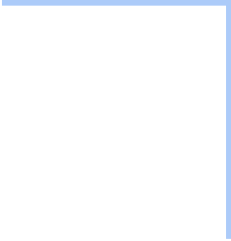
Resource Quotas



Resource Quotas

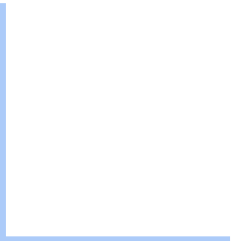
- A way to **limit** the **total amount of compute resources** that can be consumed by a set of pods in a **namespace**.
- It specifies the **maximum amount** of **CPU, memory**, and other resources that can be used by all pods in a **namespace**.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: stg-resource-quota
  namespace: ns-staging
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
    configmaps: "5"
    persistentvolumeclaims: "5"
    replicationcontrollers: "5"
    secrets: "5"
    services: "5"
    services.loadbalancers: "2"
    services.nodeports: "3"
    count/deployment.apps: "4"
```



Kubernetes Objects

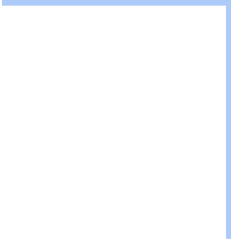
Limit Range



Limit Range

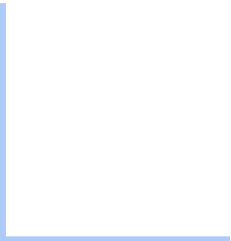
- **LimitRange** is used to set **default** and **maximum resource limits** for **individual pods** or containers within a **namespace**.
- It will only affect **newly created pods**, not existed ones.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: stg-limit-range
  namespace: ns-staging
spec:
  limits:
    - default:
        cpu: 100m
        memory: 128Mi
      defaultRequest:
        cpu: 50m
        memory: 64Mi
      max:
        cpu: 500m
        memory: 512Mi
      min:
        cpu: 50m
        memory: 32Mi
      type: Container
```



Kubernetes Objects

Services (SVC)



Service

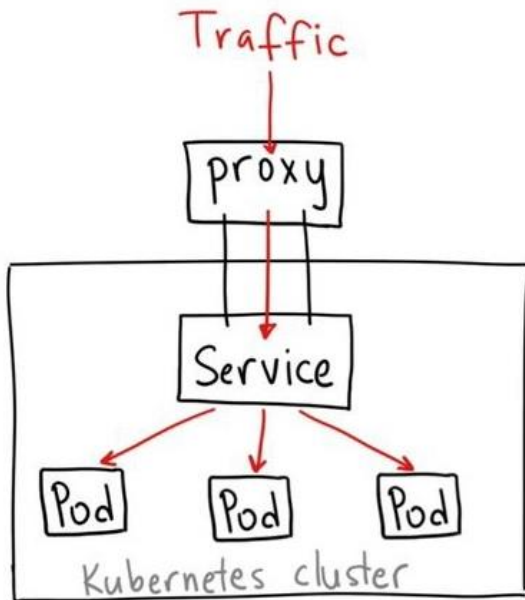
- **Services** are used to **manage networking** and **traffic flow** within a **cluster**.
- It provides a **stable IP** and **DNS name** for a set of pods.
- **Services** also enable **Load Balancing**, **Service Discovery** and **Traffic Management**.
- When a service is **created**, it is assigned a **virtual IP address** (known as a **ClusterIP**), which is used to **route traffic** to the pods that are part of the service.
- Service has a **DNS name**, which can be used to access the service from within the cluster.

`<service-name>.<namespace>.svc.cluster.local --> For Accessing a Service`
`<pod-name>.<service-name>.<namespace>.svc.cluster.local --> For Accessing a Pod`

Service Types in Kubernetes

1.ClusterIP

This is the **default type** of service, which exposes the service on a **cluster-internal IP** address that is **only** accessible from **within the cluster**.



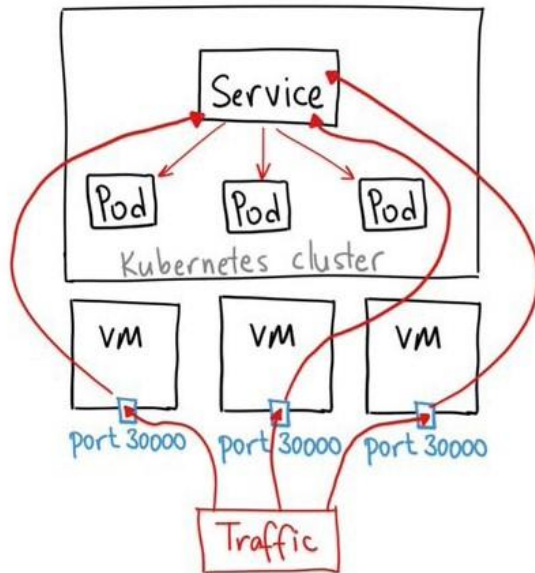
```
apiVersion: v1
kind: Service
metadata:
  name: nginx-internal
  namespace: prod
spec:
  type: ClusterIP
  selector:
    app: nginx-pod
  ports:
    - targetPort: 80
      port: 8080
```

Service Types in Kubernetes

2.NodePort

This type of service **exposes** the service on a **static port on each node** in the cluster, which can be accessed from **outside the cluster** using the node's IP address.

* NodePort must be between
30000 ~ 32767

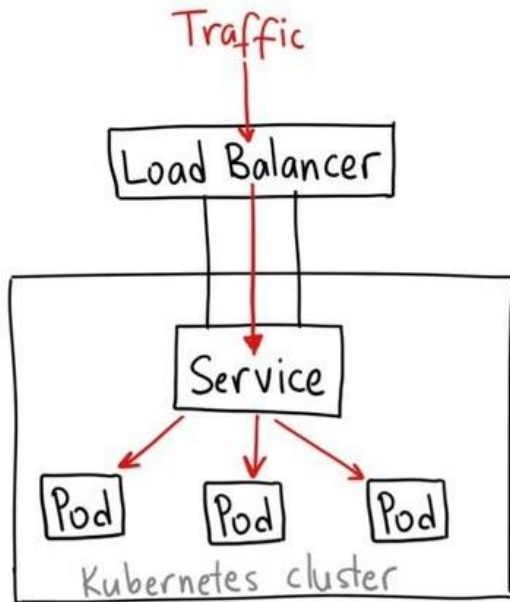


```
apiVersion: v1
kind: Service
metadata:
  name: nginx-external
  namespace: prod
spec:
  type: NodePort
  selector:
    app: nginx-pod
  ports:
    - targetPort: 80
      port: 8080
      nodePort: 30080
```

Service Types in Kubernetes

3. LoadBalancer

This type of service **exposes** the service **using a cloud provider's load balancer**, which distributes traffic to the different Pods.



```
apiVersion: v1
kind: Service
metadata:
  name: nginx-ext-lb
  namespace: prod
spec:
  type: LoadBalancer
  ports:
    - targetPort: 80
      port: 8080
      nodePort: 30080
  selector:
    app: nginx
```

Service Types in Kubernetes

4.ExternalName

This type of service maps the **service** to an **external DNS name**, allowing the service to be accessed from within the cluster using a **consistent name**.

```
apiVersion: v1
kind: Service
metadata:
  name: service-external-name-service
spec:
  type: ExternalName
  externalName: my.database.example.com
```

Pod DNS Policy

- Pod's DNS settings can be configured based on the ***dnsPolicy*** field in a Pod specification.
- This ***dnsPolicy*** field accepts three possible values:
 - **ClusterFirst:** Any DNS query that does not match the configured search domains for the Pod are forwarded to the upstream nameserver. This is the default policy if *dnsPolicy* is not specified.
 - **None:** Allows a Pod to ignore DNS settings from the Kubernetes environment. All DNS settings are supposed to be provided using the *dnsConfig* field in the Pod Spec.
 - **Default:** Use the DNS settings of the node that the Pod is running on. This means it will use the same DNS as the node that the Pod runs on.

```
apiVersion: v1
kind: Pod
metadata:
  name: dns-config-dns-config-pod
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
  dnsPolicy: "None"
  dnsConfig:
    nameservers:
      - 1.2.3.4
    searches:
      - ns1.svc.cluster-domain.example
      - my.dns.search.suffix
    options:
      - name: ndots
        value: "2"
      - name: edns0
```

Scheduling

How Scheduling Works?

- ❑ Kube-scheduler uses a set of rules to determine **which nodes** are eligible for **scheduling**. These rules include:
 - 1) **Resource requirements:** Scheduler looks at the CPU and memory requirements specified in the pod's configuration and ensures that the selected node has enough available resources to run the pod.
 - 2) **Node capacity:** The scheduler considers the capacity of each node in the cluster, including the amount of available CPU, memory, and storage, and selects a node that has sufficient capacity to meet the pod's requirements.
 - 3) **Taints and tolerations:** Nodes in a Kubernetes cluster can be tainted to indicate that they have specific restrictions on the pods that can be scheduled on them. Pods can specify tolerations for these taints, which allow them to be scheduled on the tainted nodes.

How Scheduling Works?

- 4) **Node selectors:** Users can also specify node selectors, which are labels that are applied to nodes in the cluster. The scheduler can use these selectors to filter out nodes that don't match the pod's requirements.
- 5) **(Node,Pod) Affinity/Anti-Affinity:** Kubernetes allows users to specify affinity and anti-affinity rules that control which nodes pods can be scheduled on. For example, a pod may be required to run on a node that has a specific label, or it may be prohibited from running on a node that already has a pod with a certain label.

Node Selector

```
$ kubectl label node server-02 disk=ssd
```

```
---
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: myapp-pod
```

```
spec:
```

```
  containers:
```

```
    - name: data-processor
```

```
      image: data-processor
```

```
  nodeSelector:
```

```
    disk: ssd
```

```
---
```

- A **set of labels** can assigned to a **node** for scheduling a **Pod** to run on that node.
- During creation of Pods, destination can be specified by **NodeSelector** in Pod Spec.
- **Pod** will be **run** on the nodes that **match** the **NodeSelector**.
- For more **complex** and multiple **constraints** such as **deploying** a Pod on **two nodes** with **different** labels, it's better to use **Affinity** or **Anti-Affinity**.

Affinity & Anti-Affinity

Affinity & Anti-Affinity

- **Affinity** allows to set rules based on the **node's labels** or **Pod's labels**.
- **Anti-Affinity** prevents Pods from being scheduled on the **same node** or **group of nodes**.

Affinity Type	Description
Node Affinity	Used to specify rules for which nodes a Pod can be scheduled on based on the labels of the nodes.
Pod Affinity	Used to specify rules for which Pods should be co-located on the same node based on the labels of other Pods running on the node.
Pod Anti-Affinity	Used to specify rules for which Pods should not be co-located on the same node based on the labels of other Pods running on the node.

Affinity Category	Description
Required during Scheduling	Specifies that the rule must be satisfied for the Pod to be scheduled. If the rule is not satisfied, the Pod will not be scheduled.
Preferred during Scheduling	Specifies that the rule should be satisfied for the Pod to be scheduled, but is not required. If the rule is not satisfied, the Pod will still be scheduled.

```

apiVersion: v1
kind: Pod
metadata:
  name: database-pod
spec:
  containers:
    - name: database-pod
      image: postgres:13.11
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
              - key: avail-zone
                operator: In
                values:
                  - zone1
        - weight: 20
          preference:
            matchExpressions:
              - key: share
                operator: In
                values:
                  - dedicated

```

- **Preferred Node Affinity** used to specify that the **Pod prefers** to be scheduled on **nodes** with the labels **avail-zone: zone1** and **share: dedicated**.
- **Weight** used to indicate the preference of the Pod.
- The **higher** the **weight**, the **higher** the **priority** of the label during scheduling

```

apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
spec:
  containers:
    - name: frontend-container
      image: frontend-image
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - backend
          topologyKey: "rack"

```

- **Required Pod Affinity** used to specify that the frontend-pod **requires** that to **scheduled** on a **node** with label **app=backend** and **topologyKey**.
- If **no node** has a matching Pod in the same rack, the frontend-pod **will not be scheduled**.

Taints & Tolerations

Taints & Tolerations

- **Node affinity** is a **property of pods** that can either **prefer** or **require** nodes for scheduling.
 - In contrast, **Taints** allow **nodes** to **reject** certain pods.
 - **Tolerations** are applied to **pods** and enable the scheduler to schedule them on nodes that have the **corresponding taints**.
-
- **Taints** are consist of a **key-value pair** and an effect to identify the type of **taint**.

Taint-effect:

```
$ kubectl taint nodes node-name key(=value):taint-effect
```

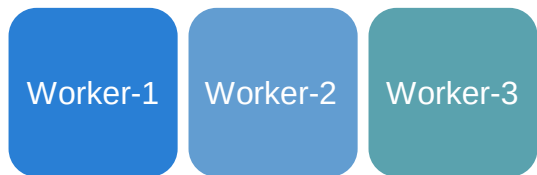
1. **NoSchedule**: It means that **no new Pods** will be **scheduled** on the Node unless they have a corresponding **toleration**. (Existing Pods on the Node will continue to run).
2. **NoExecute**: It means that **any Pods** that do **not** have a corresponding **toleration** will be **evicted** from the Node. This can be useful for where a Node needs to be **drained** or for maintenance.
3. **PreferNoSchedule**: It is **similar** to **NoSchedule**, but it **allows** Pods to be **scheduled** on the Node if there are **no other Nodes available** that match the Pod's scheduling requirements. (If there are other Nodes available that don't have taint, the Pod will be scheduled on one of those Nodes instead.)

Taints & Tolerations

- We applied a taint on workernode-3:

```
$ kubectl taint nodes worker-3 app=ssd:NoSchedule
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx-container
      image: nginx:1.18
```



nginx-pod does not tolerate the taint on the worker-3, so it will be deployed on other workers.

db-pod have this toleration, so it can be scheduled on worker-3. Also it can still be scheduled on other workers that do not have that taint.

```
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  containers:
    - name: mysql-container
      image: mysql:latest
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "ssd"
      effect: "NoSchedule"
```

Notice!

- The **node-role.kubernetes.io/master** taint is **automatically** applied by the kubelet on the **master node** when the cluster is **initialized**.
- To scheduling Pods on the master node, there are two approaches:

I. Adding a Toleration:

- By adding a toleration to the **Pod's configuration** that **matches** the **taint** on the **master node**, the Pod **can** be scheduled on the master node **despite the taint**. This allows **specific Pods** to run on the master node while preserving its **dedicated role** for control plane components and system Pods.

```
tolerations:  
  - key: "node-role.kubernetes.io/control-plane"  
    operator: "Exists"
```

II. Removing the Applied Taint:

- By this, we **opens up** master node for **scheduling** any **type of Pod**, including **regular** user Pods. However, removing taint of the master node can have **affect** on the **performance** of the cluster.

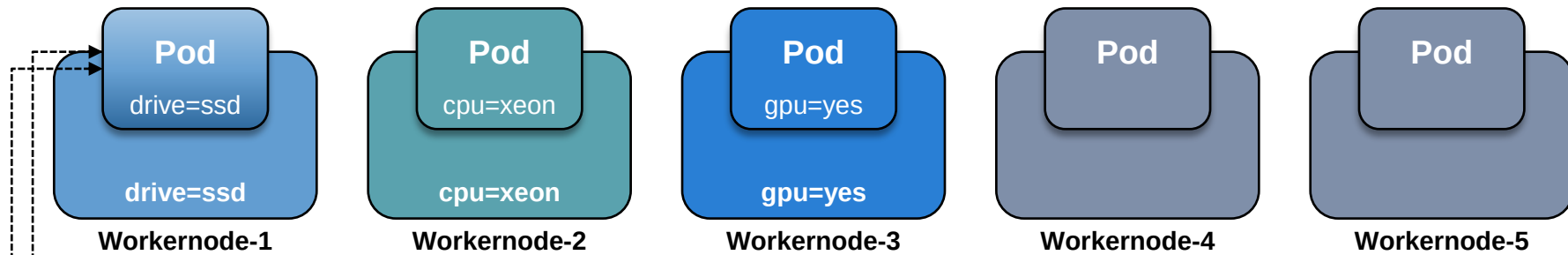
```
$ kubectl taint nodes kubemaster node-role.kubernetes.io/control-plane:NoSchedule-
```

Taints/Tolerations & Node Affinity

Taint/Tolerations & Node Affinity

- To ensure pods are scheduled on specific nodes while those nodes only accept certain pods, you can use a **combination** of **Node Affinity** and **Taints and Tolerations**.

No	Steps	Description
1	Use Node Affinity to specify the rules for selecting nodes based on their labels.	• Node Affinity is used to specify rules that determine which nodes a pod can be scheduled on. • Node affinity rules can be defined based on <u>node labels</u>, <u>node fields</u>, or <u>node selectors</u>. • By applying node affinity to a pod, you can restrict its scheduling to specific nodes that meet the defined criteria. ✓ <code>kubect1 label node kubeworker-1 drive=ssd</code> ✓ <code>kubect1 label node kubeworker-2 cpu=xeon</code> ✓ <code>kubect1 label node kubeworker-3 gpu=yes</code>
2	Use Taint/Tolerations to indicate which pods can tolerate which taints on nodes.	• Taints are applied to nodes to repel or prevent pod scheduling by default. • However, tolerations can added into the pod's configuration to allow specific pods to tolerate specific taints on nodes. • Tolerations enable pods to be scheduled on tainted nodes by matching the taint's key & value. ✓ <code>kubect1 taint nodes workernode-1 drive=ssd:NoSchedule</code> ✓ <code>kubect1 taint nodes workernode-2 cpu=xeon:NoSchedule</code> ✓ <code>kubect1 taint nodes workernode-3 gpu=yes:NoSchedule</code>



- Only pods that have the appropriate tolerations and satisfy the Node Affinity rules will be scheduled on the these nodes.
- Other nodes without the specific taint or lacking the required labels/fields won't receive these pods.

- Two Pods don't have any tolerations specified in their PodSpec, while other two nodes don't have any taints applied.
- Therefore, the scheduler can schedule these two Pods on each taintless nodes.

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: "drive"
              operator: In
              values:
                - ssd
```

```
tolerations:
  - key: "drive"
    operator: "Equal"
    value: "ssd"
    effect: "NoSchedule"
```

1

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nfs-app1
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nfs
  template:
    metadata:
      labels:
        app: nfs
    spec:
      containers:
        - name: nfs
          image: nfs:latest
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: "drive"
                    operator: In
                    values:
                      - ssd
      tolerations:
        - key: "drive"
          operator: "Equal"
          value: "ssd"
          effect: "NoSchedule"
```

2

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.17
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: "cpu"
                    operator: In
                    values:
                      - xeon
      tolerations:
        - key: "cpu"
          operator: "Equal"
          value: "xeon"
          effect: "NoSchedule"
```

3

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: image-processor-app1
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: image-processor
  template:
    metadata:
      labels:
        app: image-processor
    spec:
      containers:
        - name: image-processor
          image: image-processor:latest
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: "gpu"
                    operator: In
                    values:
                      - yes
      tolerations:
        - key: "gpu"
          operator: "Equal"
          value: "yes"
          effect: "NoSchedule"
```

Priority Class & Preemption

Priority Class & Preemption Policies

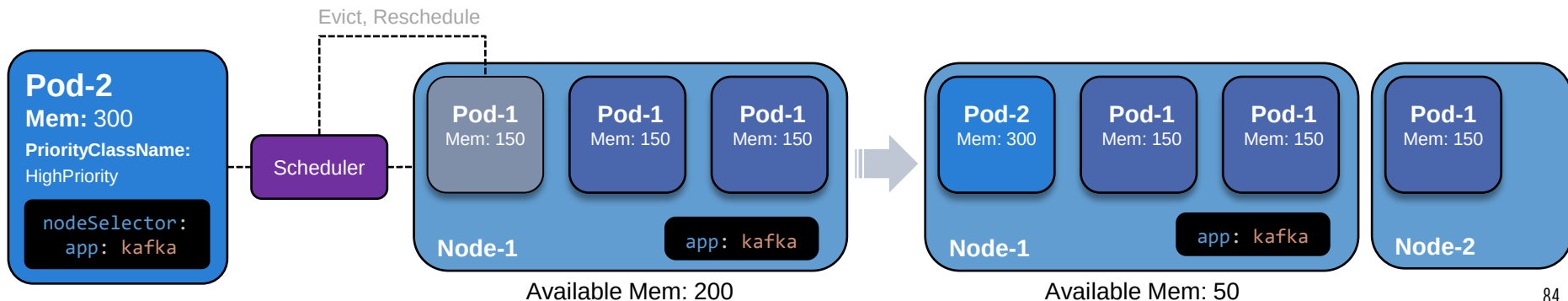
- ❑ **Priority Class** is a way to assign a **priority value** to a **Pod**. It determines its importance compared to other Pods. The priority value can be any integer between **0 ~ 1000000**, with **higher values** indicating **higher priority**.
- ❑ **Preemption Policies** determine whether a **higher priority Pod** can **preempt(evict)** a **lower priority Pod** to be scheduled on a node. There are three preemption policies:
 - I. **Preempt Lower Priority (Default):** Pods with this priority class are allowed to preempt lower priority Pods if there are no nodes with available resources to schedule them without preemption.
 - II. **IfNoOtherPods:** Pods with this priority class are allowed to preempt lower priority Pods only if there are no other Pods in the cluster that can be evicted to make room for the higher priority Pods.
 - III. **Never:** Pods with this priority class are never allowed to preempt lower priority Pods.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
```

- The **globalDefault** field indicates whether this **PriorityClass** should be used for pods without a **PriorityClass**.

```
apiVersion: v1
kind: Pod
metadata:
  name: db-1
spec:
  containers:
    - name: db-1
      image: mysql:latest
  priorityClassName: high-priority
```

- Once you have defined the **priority class**, you can **assign** it to a **Pod** by specifying its **spec.priorityClassName** field.



Pod Disruption Budget (PDB)

Pod Disruption Budget (PDB)

- **Pod Disruption Budget (PDB)** is a way to **ensure** that a **certain number** or **percentage** of pods with an application are **not voluntarily evicted** at the **same time**.
- This can help to maintain **High Availability** during voluntary disruptions like **upgrades** and **maintenance**.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: my-app
```

- In this example, the **Pod Disruption Budget** named **my-pdb** specifies that at least two pods with the label **app=my-app** should be available at all times.



DaemonSet

- **DaemonSet** is a type of controller that ensures that all (or some) nodes in a cluster run a copy of a specific pod. It is often used for system-level tasks that should be run on every node, such as log collection, monitoring, or other types of background tasks.
- When you create a DaemonSet, Kubernetes automatically creates **one pod** on **each node** that **matches** the specified **label selector**.
- If a new node added to cluster, Kubernetes **automatically** creates a **new pod** on that node as well.
- **By using labels and node selectors, you can specify which nodes in the Kubernetes cluster should run a particular DaemonSet.**

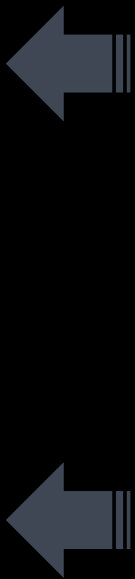
What is the best way to have a DaemonSet without consuming too many resources from customer's service?

DaemonSet

The selector section specifies the **label selector** used to identify which pods are managed by the **DaemonSet**.

Pod of the DaemonSet will be created on the nodes that have label **Drive=SSD**.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
  namespace: dev-drive-monitor
spec:
  selector:
    matchLabels:
      app: monitoring
  template:
    metadata:
      labels:
        app: monitoring
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
      nodeSelector:
        Drive: ssd
```



AutoScaling

- **Autoscaling** is the ability of Kubernetes to **automatically** adjust the **number of running instances** of a specific workload based on the **current demand** or **load**.
- It helps to ensure there are **enough resources** available to handle the workload while also **optimizing resource utilization**.
- Kubernetes provides **two types of autoscaling mechanisms**:

HPA

(Horizontal Pod AutoScaling)

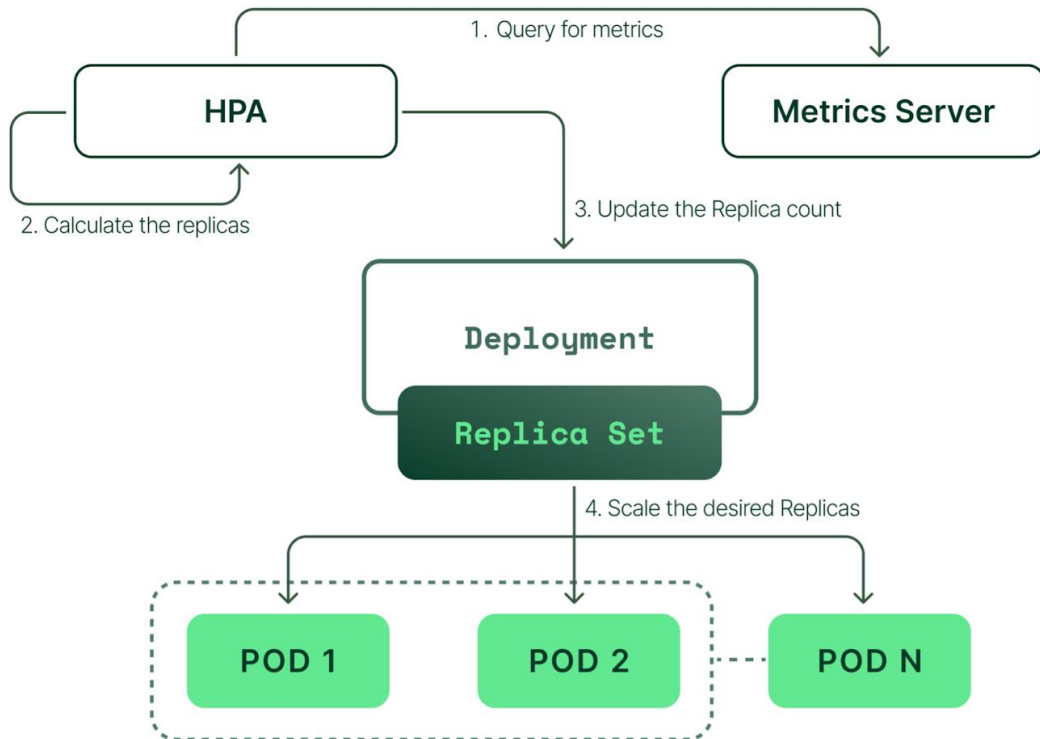
VPA

(Vertical Pod AutoScaling)

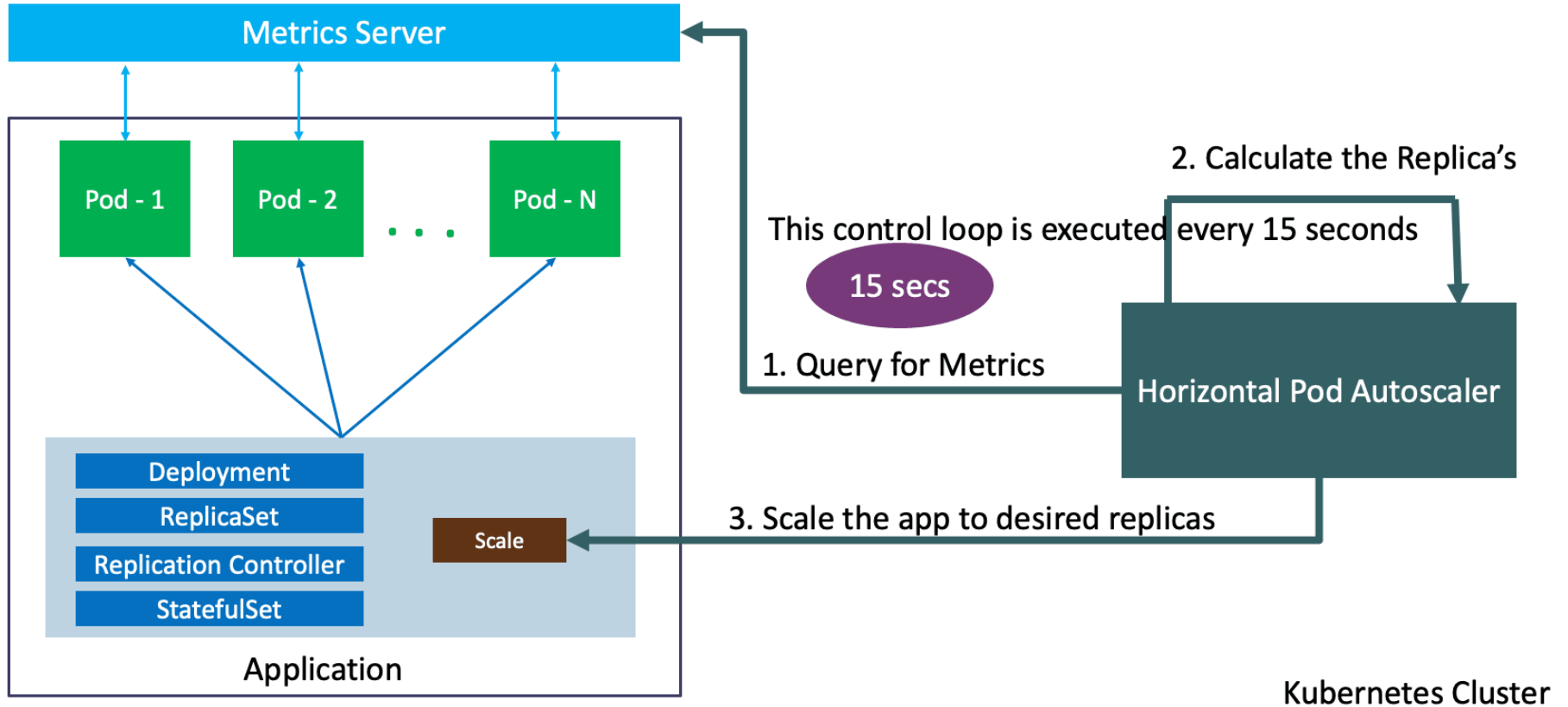
HPA

✓ *Scaling out, also known as horizontal scaling, it is the process of adding more replicas of a Deployment or ReplicaSet to handle an increase in traffic or load.*

- **HPA** allows you to automatically **scale out** your application by **adding or removing replicas** based on **resource utilization** metrics such as **CPU utilization** or custom metrics.
- This ensures that you have **necessary resources** to handle **increased traffic** load.



How HPA Works?



1.Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
  namespace: dev
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
        - name: php-apache
          image: k8s.gcr.io/hpa-example
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: 500m
            requests:
              cpu: 200m
```

2.Service

```
apiVersion: v1
kind: Service
metadata:
  name: php-apache
  namespace: dev
  labels:
    run: php-apache
spec:
  selector:
    run: php-apache
  ports:
    - port: 80
      targetPort: 80
```

HPA Example!

3.HPA

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  targetCPUUtilizationPercentage: 50
```

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
  namespace: dev
spec:
  scaleTargetRef:
    kind: Deployment
    name: php-apache
    apiVersion: apps/v1
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 40
```

```
behavior:
  scaleUp:
    policies:
      - type: Pods
        value: 5
        periodSeconds: 30
      - type: Percent
        value: 100
        periodSeconds: 30
    selectPolicy: Max
    stabilizationWindowSeconds: 40
  scaleDown:
    policies:
      - type: Pods
        value: 4
        periodSeconds: 10
      - type: Percent
        value: 10
        periodSeconds: 10
    selectPolicy: Min
    stabilizationWindowSeconds: 5
```

3.HPA

VPA Example!

- **VPA** automatically **scale up** or **down** the **resource requests** and limits of containers in a Pod based on actual resource usage.
- It ensures each Pod has **necessary resources** to handle the workload **efficiently**.

✓ *Scaling up, also known as vertical scaling, is the process of increasing the resources available to each replica of a Deployment or ReplicaSet to handle an increase in demand*

1.Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
  namespace: dev
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
        - name: php-apache
          image: k8s.gcr.io/hpa-example
          ports:
            - containerPort: 80
      resources:
        requests:
          cpu: "20m"
          memory: "200Mi"
        limits:
          cpu: "500m"
          memory: "1Gi"
```

```
apiVersion: "autoscaling.k8s.io/v1"
kind: VerticalPodAutoscaler
metadata:
```

```
  name: php-apache
  namespace: dev
```

```
spec:
```

```
  targetRef:
```

```
    apiVersion: "apps/v1"
    kind: Deployment
    name: php-apache
```

```
  updatePolicy:
```

```
    updateMode: "Auto"
```

```
  resourcePolicy:
```

```
    containerPolicies:
```

```
      - containerName: '*'
        mode: "Auto"
        controlledValues: "RequestsAndLimits"
        minAllowed:
          cpu: 10m
          memory: 5Mi
        maxAllowed:
          cpu: 200m
          memory: 500Mi
```

```
  controlledResources: ["cpu", "memory"]
```

2.VPA

```
apiVersion: "autoscaling.k8s.io/v1"
kind: VerticalPodAutoscaler
metadata:
```

```
  name: php-apache
  namespace: dev
```

```
spec:
```

```
  targetRef:
```

```
    apiVersion: "apps/v1"
    kind: Deployment
    name: php-apache
```

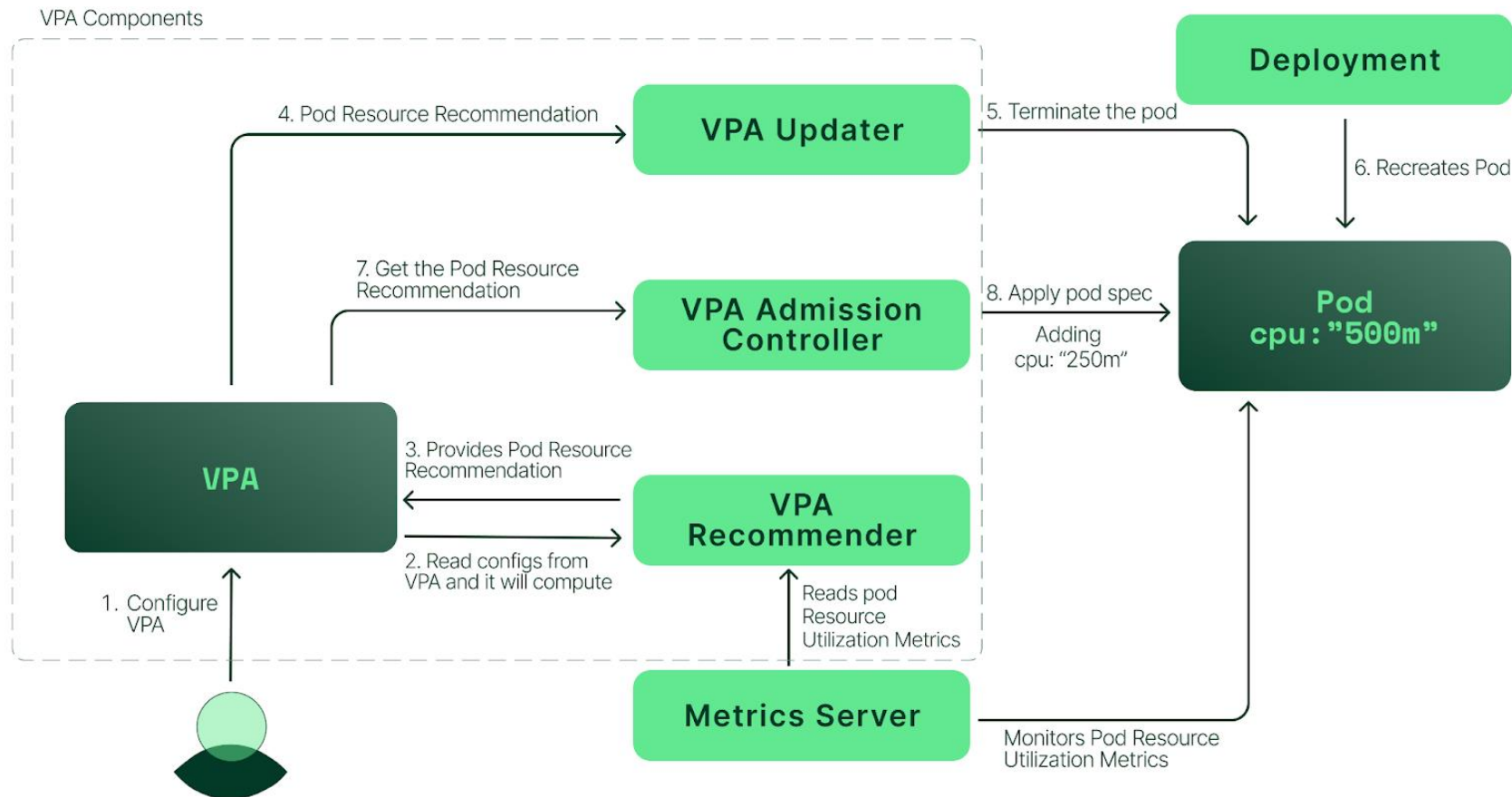
```
  updatePolicy:
```

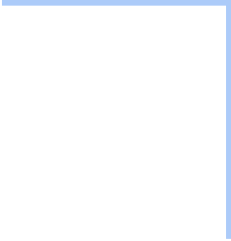
```
    updateMode: "off"
```

2.VPA

- If **UpdateMode** turns **off**, it means you must need to configure **CPU/Mem parameters** in your Manifest files by manual.
- With the **Auto UpdateMode**, it will **automatically** update resource **requests** and **limits** of the containers in the pods.

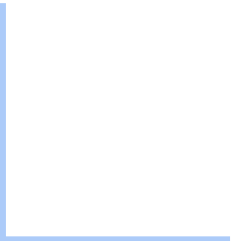
How VPA Works?





Configure Application

ConfigMap, Environment Variables, Secret



ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-configmap
data:
  config.yaml: |
    DB_HOST: "mydbhost"
    DB_PORT: "5432"
    DB_NAME: "mydb"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: mycontainer
      image: myimage
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-configmap
```

- **ConfigMaps** are Kubernetes resources that used for store **configuration data** as **key-value pairs**.
- A **ConfigMap** will be created with **desired config data** and then referenced in **Deployments** or **Pod Specs** using **configMapKeyRef** field or **mount** it directly to the Pod.
- It can be created using the **kubectl create configmap** command, or by **defining a YAML file**.

Environment Variables

- **Environment variables** can be used to pass **configuration information to the container**, such as **database connection strings** or **API keys**.
- It can be defined using **'env'** field, **'envFrom'** field, and **'valueFrom'** field.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: web
          image: myapp:latest
          ports:
            - containerPort: 80
          env:
            - name: DB_HOST
              value: "server2"
            - name: DB_PORT
              value: "5432"
            - name: DB_NAME
              value: "payments"
```

TYPE I "env"

Environment Variables

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: db-config
data:
  DB_HOST: "server2"
  DB_PORT: "5432"
  DB_NAME: "payments"
```

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secrets
type: Opaque
stringData:
  DB_USER: dXNlcg=sas
  DB_PASSWORD: cGFzc3dvcmQ
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: web
          image: myapp:latest
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: db-config
            - secretRef:
                name: db-secrets
```

TYPE II
"envFrom"

Environment Variables

- **'valueFrom'** field is used to define **environment variables** for a container, **based on a field** in another resource, such as a **ConfigMap** or **Secret**.
- Name of the resource can be specified and the field using the **'configMapKeyRef'** or **'secretKeyRef'** fields, respectively

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: db-config
data:
  db-host: server2
  db-port: "5432"
  db-name: "payments"
```

TYPE III "valueFrom"

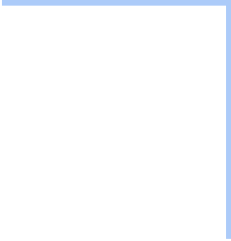
```
apiVersion: apps/v1
kind: Deployment
...
template:
  metadata:
    labels:
      app: myapp
  spec:
    containers:
      - name: web
        image: myapp:latest
        ports:
          - containerPort: 80
        env:
          - name: DB_HOST
            valueFrom:
              configMapKeyRef:
                name: db-config
                key: db-host
          - name: DB_PORT
            valueFrom:
              configMapKeyRef:
                name: db-config
                key: db-port
```

Secrets

- **Secrets** are similar to **ConfigMaps**, but are used to store **sensitive information** such as passwords, tokens or API keys.
- A **Secret** can be created with desired **sensitive information**, and referenced in Deployments spec using the '**secretKeyRef**' field.

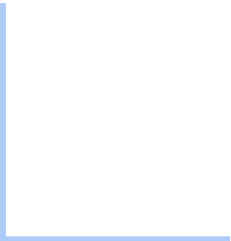
□ There are **several types** of secrets in Kubernetes, including:

1. **Opaque:** Default secret type in K8s. It's used to store any **arbitrary data** and is **encoded** in **base64**.
2. **TLS:** It is used to store **TLS certificates** and **keys**. It contains two keys: **tls.crt** and **tls.key**.
3. **Docker-registry:** It is used to **authenticate** with a **Docker registry**. It contains user/pass of registry.
4. **SSH:** It is used to store **SSH keys**. It contains **private key** and **public key**.
5. **Service account:** It is **automatically created** by K8s when a **service account** is **created**. It contains a **token** that can be used to **authenticate** the service account.



Lifecycle Management

Init Container, Sidecar Container



Init Container

- **Init Container** is a container that **runs before** the **main container(s)** in a **pod**.
- Its purpose is to perform some **pre-configs** or **setup tasks** that are required.
- It is defined in the **same YAML** file as the **pod specification**.
- Init Container has a **lifecycle**, and it will be **successful** if completed its tasks **without error**.
- If an Init Container **fails**, K8s will **attempt to restart** it until it succeeds.

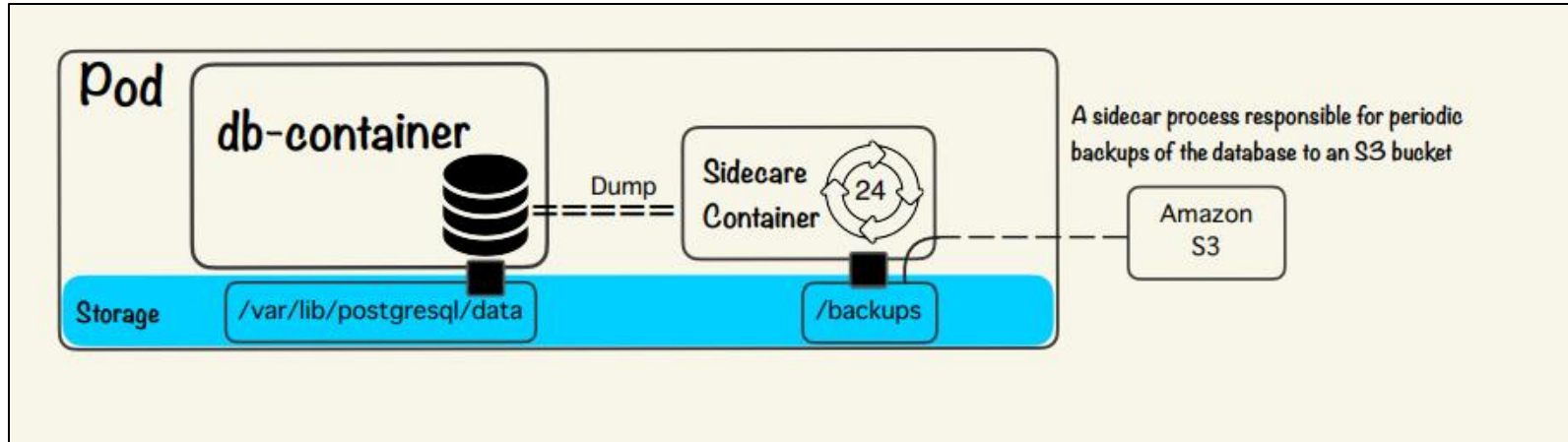
```
apiVersion: v1
kind: Pod
metadata:
  name: my-webapp-pod
spec:
  initContainers:
    - name: redis-setup
      image: redis:latest
      command: ["sh", "-c"]
      args:
        - |
          redis-cli ping || exit 1
          redis-cli config set maxmemory 1gb
          redis-cli config set save ""
  containers:
    - name: webapp
      image: my-webapp-image
      ports:
        - containerPort: 80
      env:
        - name: REDIS_HOST
          value: redis-service
        - name: REDIS_PORT
          value: "6379"
```

Init Container

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql-db
spec:
  containers:
    - name: mysql
      image: mysql:5.7
      env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secrets
              key: password
  initContainers:
    - name: migrate-db
      image: mysql:5.7
      command: ['sh', '-c', 'mysql -h ${DB_HOST} -u root -p ${DB_PASSWORD} ${DB_NAME} < /migrations/migrate.sql']
      env:
        - name: DB_NAME
          value: mydb
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secrets
              key: password
      volumeMounts:
        - name: migrations
          mountPath: /migrations
  volumes:
    - name: migrations
      configMap:
        name: db-migrations
```

Sidecar Container

- **Sidecar Container** is a **container** that is deployed **alongside** a **main container** in a **pod**.
- Usually **Sidecar Container** provides **support** or **complementary functionality** to the **main container**.
- Use cases: Logging & Monitoring, Backup & Recovery, Service Mesh, etc.



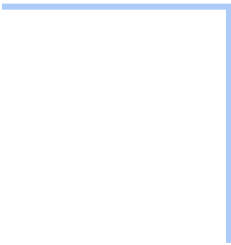
Sidecar Container

```
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  containers:
  ## Main Container
  - name: db-container
    image: my-database-image
    env:
      - name: DATABASE_URL
        value: "postgresql://my-database-
hostname:5432/my-database"
    ports:
      - containerPort: 5432
    volumeMounts:
      - name: db-data
        mountPath: /var/lib/postgresql/data
  ## Sidecar Container
  - name: sidecar-container
    image: my-sidecar-image
    env:
      - name: BACKUP_LOCATION
        value: "s3://my-bucket/my-backups"
      - name: DATABASE_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secrets
            key: database-password
```

-->

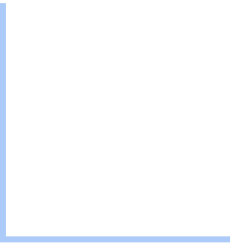
-->

```
volumeMounts:
  - name: backup-data
    mountPath: /backups
  - name: db-secrets
    mountPath: /secrets
command: ["/bin/sh", "-c"]
args:
  - |
    while true; do
      pg_dump -U postgres -h localhost my-database |
gzip > /backups/mydatabase-$(date +%Y-%m-%d-
%H%M%S).sql.gz; s3cmd put /backups/my-database-*.sql.gz
"$BACKUP_LOCATION";
      sleep 86400;
      s3cmd put /backups/my-database-*.sql.gz
"$BACKUP_LOCATION";
    done
volumes:
  - name: db-data
    emptyDir: {}
  - name: backup-data
    emptyDir: {}
  - name: db-secrets
    secret:
      secretName: db-secrets
```



Lifecycle Management

Jobs, CronJobs



Jobs

- **Job** is resource that allows to **create** and **manage a finite or batch process** in the cluster.
- It's commonly used for **tasks** that need to be **run once** or **a few times**, such as data processing, backups, or migrations.

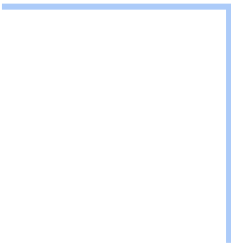
```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processing-job
spec:
  backoffLimit: 3
  template:
    spec:
      containers:
        - name: data-processor
          image: data-processor:v1.4
          command: ["python", "process_data.py"]
          restartPolicy: Never
```

The **backoffLimit** specifies the number of times k8s should retry the Job if it fails before giving up.

CronJobs

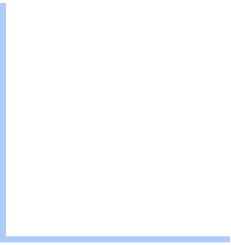
- **CronJobs** is a way to **schedule** and **automate execution of Jobs** on a recurring basis.
- A **Job** creates **one or more Pods** to perform a **specific task**, but a **CronJob** allows **Jobs** to be **scheduled** according to a **specific time or interval**, similar to **Unix Cron** utility.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report-generation-cronjob
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          ttlSecondsAfterFinished: 100
          containers:
            - name: report-generator
              image: my-django-app:v1
              env:
                - name: DJANGO_SETTINGS_MODULE
                  value: myapp.settings
              command: ["python", "manage.py", "generate_report"]
          restartPolicy: Never
```



Self-Healing Probes

Liveness, Readiness, Startup Probes



Self-Healing Probes

Liveness Probe

- Kubernetes uses liveness probes to know when to restart a Container. For example, a liveness probe could catch a deadlock, where an application is running, but unable to make progress. Restarting a Container in such a state can help to make the application more available despite bugs.

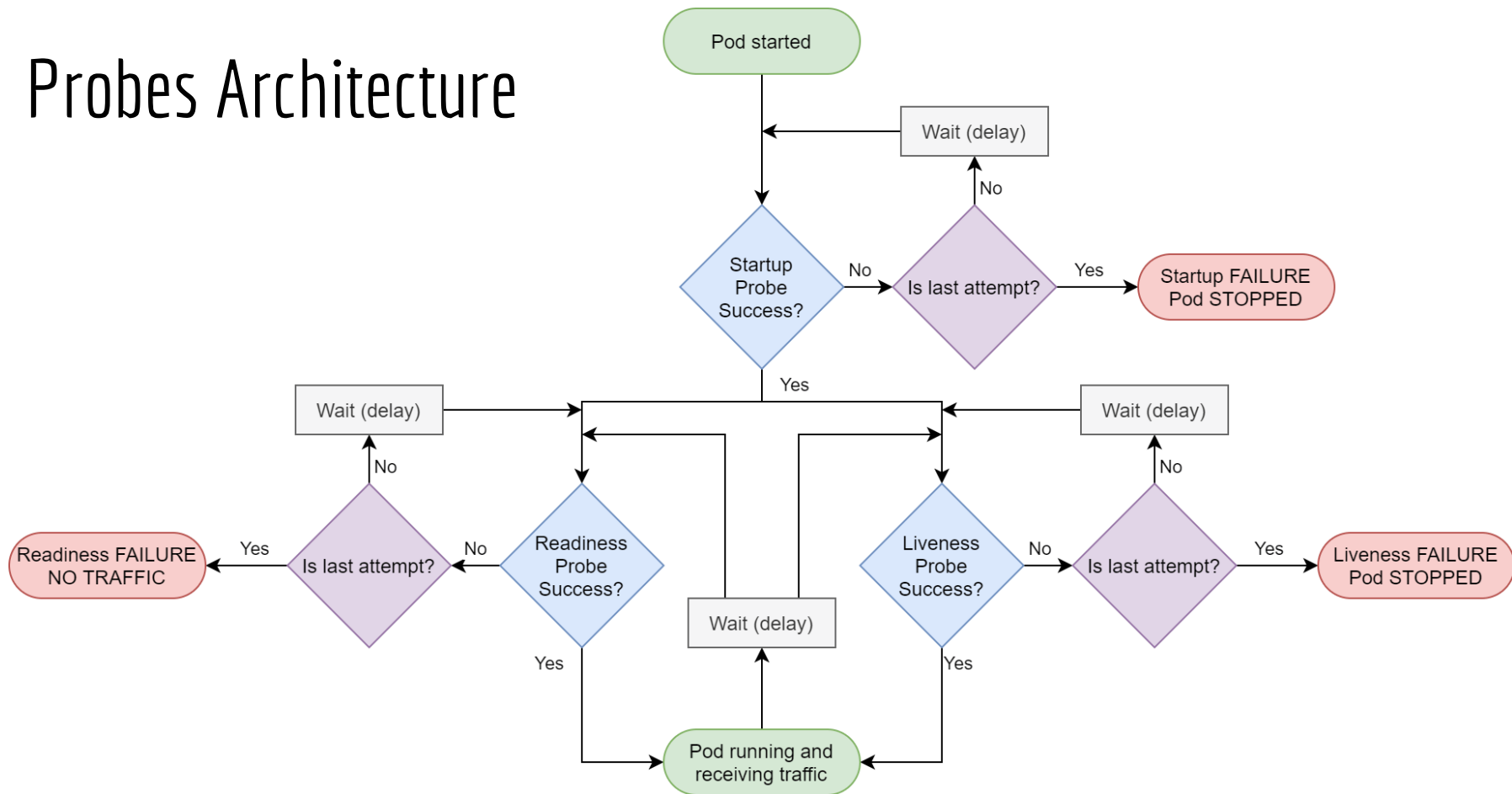
Readiness Probe

- Kubernetes uses readiness probes to know when a Container is ready to start accepting traffic. A Pod is considered ready when all of its Containers are ready. One use of this signal is to control which Pods are used as backends for Services. When a Pod is not ready, it is removed from Service load balancers.

Startup Probe

- These probes let Kubernetes know when your application has started. If such a probe is configured, it disables liveness and readiness checks until it succeeds, making it useful for slowstarting containers.

Probes Architecture



Self-Healing Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: my-image
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
```

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: my-image
    ports:
    - containerPort: 8080
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 10
      failureThreshold: 3
```

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: my-image
    livenessProbe:
      exec:
        command:
        - /bin/sh
        - -c
        - /usr/bin/script.sh
      initialDelaySeconds: 30
      periodSeconds: 10
```

Backup & Restore Methods

Backup Using etcdctl

- You can backup Kubernetes resources using **etcdctl** command line.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: backup-pvc
spec:
  storageClassName: <storage-class>
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: etcd-backup
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: etcd-backup
              image: quay.io/coreos/etcd:v3.5.0
              command:
                - /bin/sh
                - -c
                - |
                  ETCDCTL_API=3 etcdctl snapshot save /backup/k8s/etcd-snapshot.db \
                  --endpoints=<ETCD-endpoints> \
                  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
                  --cert=/etc/kubernetes/pki/etcd/server.crt \
                  --key=/etc/kubernetes/pki/etcd/server.key
          volumeMounts:
            - name: etcd-certs
              mountPath: /etc/kubernetes/pki/etcd
            - name: backup
              mountPath: /backup
          volumes:
            - name: etcd-certs
              secret:
                secretName: etcd-certs
            - name: backup
              persistentVolumeClaim:
                claimName: backup-pvc
```

- This is a **CronJob** that runs **etcd backup** job **every hour**, using the **etcdctl** tool inside a container to create a snapshot of **etcd database** and save it to the specified path.
- It mounts the **etcd certificates** and a **PersistentVolumeClaim** for storing the backup.

Restore etcd Backup Using etcdctl

1. Ensure **kube-api-server** on your new cluster is **not active**.
2. Change the path of **etcd data directory** to **/var/lib/etcd-backup/** directory. For this you need to **edit** the manifest file of etcd and update its **hostPath specification**.
3. Use **etcdctl** to restore the backup:

```
$ etcdctl snapshot restore /backup/k8s/etcd-snapshot.db \  
  --data-dir=/var/lib/etcd-from-backup \  
  --initial-cluster=etcd01=kubemaster-1=https://192.168.100.11:2380,etcd02=http://<etcd02-ip>:2380 \  
  --initial-advertise-peer-urls=https://192.168.100.11:2380 \  
  --initial-cluster-token=<ETCD-initial-cluster-token> \  
  --name=kubemaster-1
```

Backup Using Velero

- You can create and restore backups using both velero command-line tool or creating a **CronJob** to schedule getting backup each hour.

```
$ velero backup create <backup-name> \  
  --include-namespaces my-namespace \  
  --include-resources deployments,pods
```

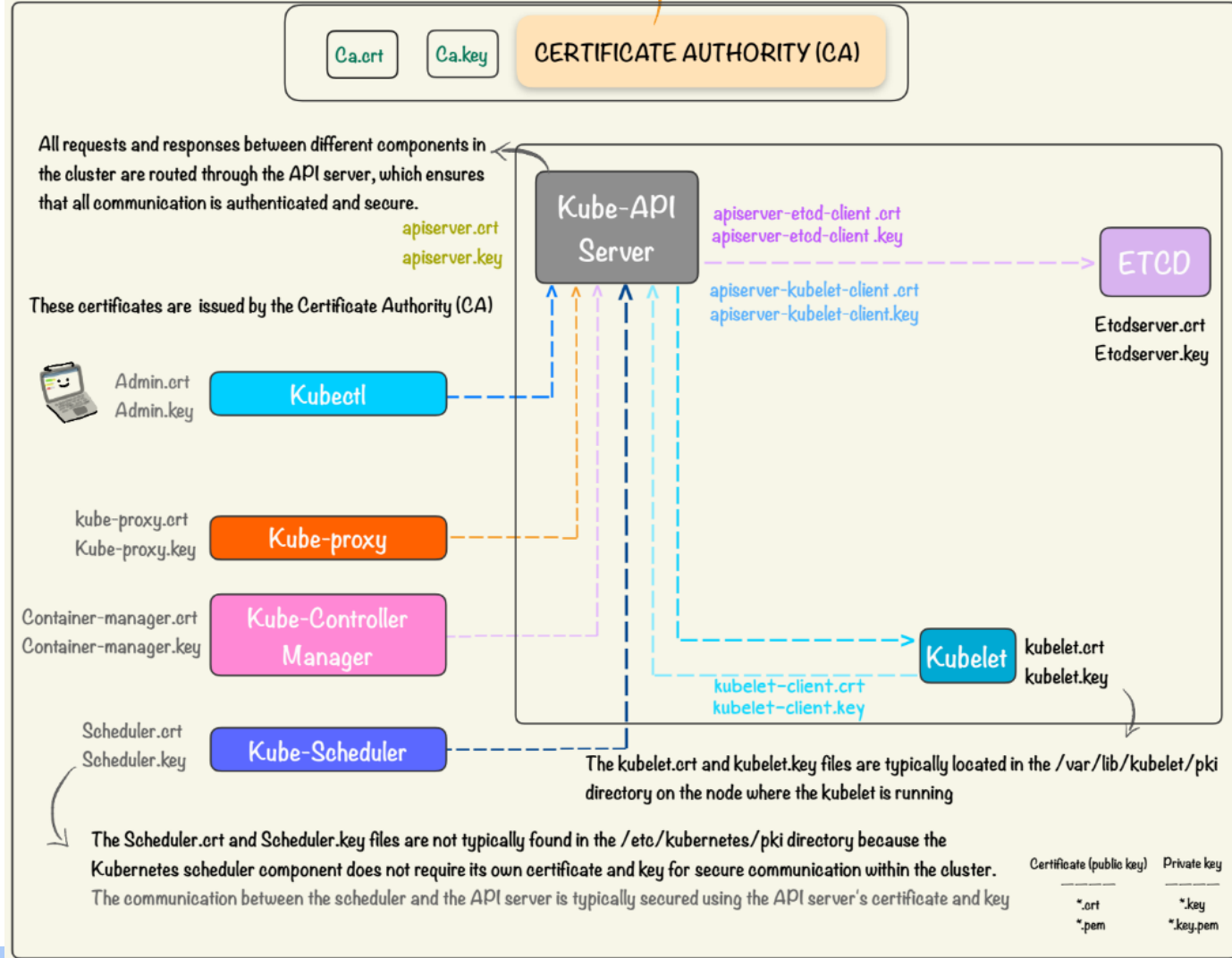
```
$ velero restore create \  
  --from-backup <backup-name> \  
  --include-namespaces my-namespace \  
  --include-resources deployments,pods
```

```
apiVersion: batch/v1beta1  
kind: CronJob  
metadata:  
  name: velero-backup  
spec:  
  schedule: "0 * * * *"  
  jobTemplate:  
    spec:  
      template:  
        spec:  
          containers:  
            - name: velero  
              image: velero/velero:v1.7.0  
              command:  
                - /velero  
              args:  
                - backup  
                - create  
                - my-backup  
              volumeMounts:  
                - name: cloud-credentials  
                  mountPath: /credentials  
              env:  
                - name: AWS_ACCESS_KEY_ID  
                  valueFrom:  
                    secretKeyRef:  
                      name: cloud-credentials  
                      key: aws_access_key_id  
                - name: AWS_SECRET_ACCESS_KEY  
                  valueFrom:  
                    secretKeyRef:  
                      name: cloud-credentials  
                      key: aws_secret_access_key  
          volumes:  
            - name: cloud-credentials  
              secret:  
                secretName: cloud-credentials
```

Security

- /etc/kubernetes/
- /etc/kubernetes/pki/

Security Architecture in Kubernetes

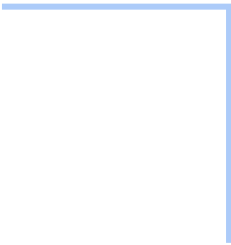


Certificates in Kubernetes

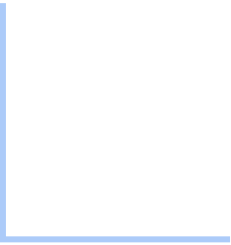
1. It's advised to **backup** all **old certificates** before renewing:
 - `/etc/kubernetes/*.conf`
 - `/etc/kubernetes/pki/*`
 - `~/.kube/config`
2. Certificates generated by **Kubeadm** will **expire** after **1 year** and need to be **renewed**:

```
$ openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout
$ kubeadm certs check-expiration
$ kubeadm certs renew all
```

3. After renewing certificates, need to **restart Control-Plane pods** (Static pods). For this, you cannot use *kubectl*, you can temporarily remove its manifest from `/etc/kubernetes/manifests/`.
4. If you upgrade your kubeadm version, certificates also will be renewed.

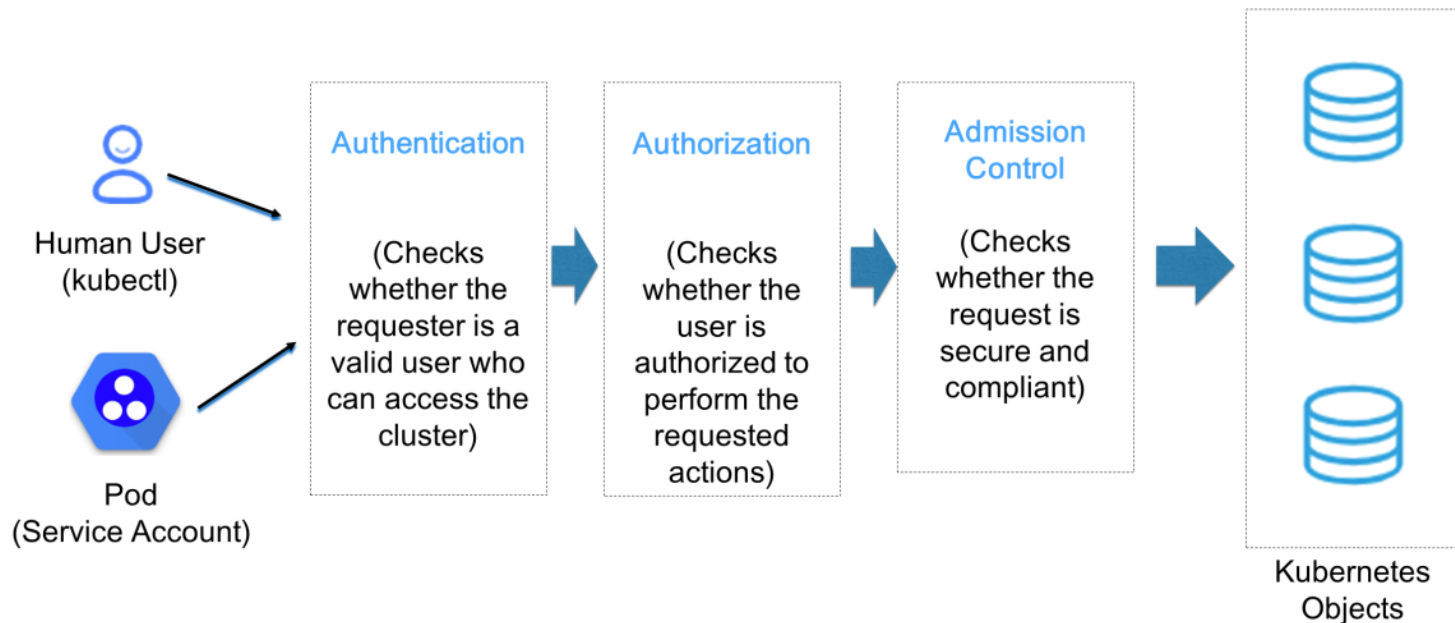


Authentication, Authorization, Admission Control



Authentication, Authorization & Admission Control

- When a client (such as kubectl) sends an **API request** to Kubernetes, the request goes through several steps before it is processed and a response is sent back to the client.



Authentication

- Kubernetes uses **authentication mechanisms** to verify **identity of users** and **components** trying to access the cluster. This ensures that only authorized users and components can access the cluster.

Auth Methods	Description
Service Account Authentication	This is a method that uses Kubernetes Service Accounts to authenticate clients. Each Service Account in the cluster has its <u>own token</u> that is used to authenticate clients. This method is commonly used when a cluster has a <u>large number of clients</u> or when <u>automated processes</u> need to access the Kubernetes API.
X.509 Client Certificates Authentication	This is a method that uses digital certificates to authenticate clients. Each client in the cluster has its <u>own certificate</u> and <u>private key</u> that are used to authenticate and encrypt communication when communicating with other components. This method is commonly used when a cluster has a <u>small number of clients</u> or when <u>strong authentication</u> is required.
Webhook Authentication	This is a method that uses an external HTTP service to authenticate clients. It is commonly used when a cluster needs to <u>integrate</u> with a <u>custom authentication system</u> . The external HTTP service receives authentication requests from the Kubernetes API server and returns a response indicating whether the client is authenticated or not.

Authorization

- It means **what actions** a **user** or **component** can perform within the cluster. This ensures that users and components have access only to the **resources** they are **authorized to access**.

Auth Methods	Description
Role-Based Access Control (RBAC) RBAC is a mechanism that allows to control access to resources based on user's role & permissions. It needs to define Roles , ClusterRoles , RoleBindings and ClusterRoleBindings .	
Role	Defines permissions for resources within a specific namespace. Think of it as a set of rules for what actions can be done in that namespace.
RoleBinding	Connects a Role to a user or group within a namespace, giving them the permissions defined in the Role.
ClusterRole	Similar to a Role, but it applies to the entire cluster, not just a single namespace.
ClusterRole Binding	Connects a ClusterRole to a user or group across the whole cluster, granting them the permissions defined in the ClusterRole.

Examples!

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: secret-reader
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "watch", "list"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: "jane"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-secrets-global
subjects:
- kind: User
  name: "jane"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io
```

Admission Control

- Kubernetes uses an **Admission control mechanism** for enforcing **rules** and **policies** on resources **before** they are created or updated in the cluster. This can include validating the structure and content of resource manifests, applying **default values**, and **enforcing constraints** on resource usage. There are two types of admission control plugins:

Admission Control	Description
Validating Admission Plugins	These plugins validate the request object without modifying it. They can reject the request if it doesn't meet the required criteria.
Mutating Admission Plugins	These plugins can modify the request object, as well as validate it. They are executed before the validating admission plugins.

Service Account

Service accounts in Kubernetes are **non-human accounts** that provide a **unique identity** for system components and application pods.

These accounts are **namespace-specific objects** managed within the **kube-api-server**.

By default, **each namespace** includes a **service account** called "**default**" which has no special roles or privileges assigned to it.

Every **Namespace** has a **Default Service Account**, which pods without specifying Service Account, will be use default one.

From Kubernetes **1.24** onwards, **automatic token generation** has been **discontinued**, and **tokens** must be acquired through the **TokenRequest API** or by creating a **Secret API object**, allowing the token controller to populate it with a service account token.

- With using `kubernetes.io/service-account.name` K8s automatically provide a **Secret** with a Service Account **Token** for it and Token Controller will take care of it.

```
$ kubectl create serviceaccount monitoring-agent-SA
```

0

```
apiVersion: v1
kind: Secret
metadata:
  name: monitoring-agent-SA-token
  annotations:
    kubernetes.io/service-account.name: monitoring-agent-SA
type: kubernetes.io/service-account-token
```

1

- **TokenRequest** allows to request a token for a specific **Service Account**. It will create a short-lived tokens for authentication & authorization purposes.

```
apiVersion: authentication.k8s.io/v1
kind: TokenRequest
metadata:
  name: monitoring-agent-SA-token
spec:
  audiences:
    - rbac.authorization.k8s.io
  serviceAccountName: monitoring-agent-SA
```

2

- The parameters that can be written in "audiences" :
- `rbac.authorization.k8s.io` -- `metrics.k8s.io` -- `authentication.k8s.io` -- `storage.k8s.io` -- `api`

- Create a **ClusterRole** that grants the necessary permissions for SA and Create a **ClusterRoleBinding** that associates the **ServiceAccount** with the **ClusterRole**.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: monitoring-agent-role
rules:
  - apiGroups: [""]
    resources: ["pods", "nodes"]
    verbs: ["get", "list", "watch"]
```

3

- To check the permissions of a ServiceAccount, execute the following command to list all the available permissions granted to the monitoring-agent-SA ServiceAccount in the default namespace:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: monitoring-agent-role-binding
subjects:
  - kind: ServiceAccount
    name: monitoring-agent-SA
roleRef:
  kind: ClusterRole
  name: monitoring-agent-role
apiGroup: rbac.authorization.k8s.io
```

4

```
$ kubectl auth can-i --list --as=system:serviceaccount:default:monitoring-agent-SA
```

- When you specify the `serviceAccountName` field in the Pod spec, Kubernetes mounts the secret containing the Service Account token as a volume in the Pod.
- The volume is mounted at `/var/run/secrets/kubernetes.io/serviceaccount`, and the Service Account token is stored in the token file inside this volume

```
apiVersion: v1
kind: Pod
metadata:
  name: monitoring-agent
spec:
  serviceAccountName: monitoring-agent-SA
  containers:
  - name: monitoring-agent
    image: monitoring-agent-image
    args: ["--kubeconfig=/var/run/secrets/kubernetes.io/serviceaccount/token"]
    volumeMounts:
    - name: sa-token
      mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      readOnly: true
  volumes:
  - name: sa-token
    secret:
      secretName: monitoring-agent-SA-token-xxxxx
```

5

Storage

HostPath, EmptyDir, ConfigMaps, Secrets, PVs, PVCs

Storage: *HostPath Volume*

- **HostPath volumes** allow you to **mount** a **directory** from the **host node's filesystem** into a **Pod**.
- This approach is **useful** for **testing** and **development** purposes, but it is **NOT recommended** for **production environments** as it can create security risks.

```
apiVersion: v1
kind: Pod
metadata:
  name: web-app
  namespace: dev
spec:
  containers:
    - name: web-app
      image: my-web-app-image
      volumeMounts:
        - mountPath: /app
          name: user-uploads
  volumes:
    - name: user-uploads
      hostPath:
        path: /tmp/app
        type: DirectoryOrCreate
```

Storage: *EmptyDir Volume*

- **EmptyDir volumes** are a type of **temporary storage** that are created and attached to a Pod when the Pod is created.
- The data stored in an **EmptyDir volume** exists only for the **lifetime of the Pod** and is **deleted** when the Pod is deleted.
- These volumes are commonly used for storing **temporary data** that is needed by a Pod, such as **cache files** or **temporary log**.

```
apiVersion: v1
kind: Pod
metadata:
  name: ML-app
spec:
  containers:
    - name: video-conv
      image: video-conv
      volumeMounts:
        - name: cache-volume
          mountPath: /var/cache/data
  volumes:
    - name: cache-volume
      emptyDir:
        medium: Memory
        sizeLimit: 1Gi
```

```
apiVersion: v1
kind: Pod
metadata:
  name: monit-pod
spec:
  containers:
    - name: monit-container
      image: monit-image
      volumeMounts:
        - name: logs-volume
          mountPath: /var/log/mon-app
  volumes:
    - name: logs-volume
      emptyDir:
        sizeLimit: 1Gi
```

Storage: *Persistent Volume (PV) & Persistent Volume Claim (PVC)*

- **PVs** are independent volumes that can be provisioned from different **storage providers** such as **cloud storage** or **on-premise storage systems**.
- **PVCs** are used to **request storage** resources from the **PVs**.
- You can use PVs and PVCs to store data persistently, even if a Pod is deleted or restarted. PVs and PVCs can be used with **different storage backends** like **NFS, iSCSI, Ceph**, etc

Access Modes	CLI Name	Description
ReadWriteOnce	RWO	The volume can be mounted as read-write by a single node.
ReadOnlyMany	ROX	The volume can be mounted read-only by many nodes.
ReadWriteMany	RWX	The volume can be mounted as read-write by many nodes.
ReadWriteOncePod	RWOP	the volume can be mounted as read-write by a single Pod. Use ReadWriteOncePod access mode if you want to ensure that only one pod across whole cluster can read that PVC or write to it.

Storage: *Persistent Volume (PV) & Persistent Volume Claim (PVC)*

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv1-40g-rw
spec:
  capacity:
    storage: 40Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: nfs_server_ip
    path: /mnt/nfs_share/pv1-40g-rw
  persistentVolumeReclaimPolicy: Retain
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc-20g
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 15Gi
```

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: my-image
      volumeMounts:
        - name: my-volume
          mountPath: /data
  volumes:
    - name: my-volume
      persistentVolumeClaim:
        claimName: nfs-pvc-20g
```

StatefulSet

StatefulSet

- **StatefulSets** are used to manage stateful applications in Kubernetes.
- It'll handle applications that require **unique identities, stable network addresses, persistent storage, ordered deployment** and **scaling**, and **graceful deletion**. Such as databases & MQs.
- It'll maintain a **sticky identity** for **each pod**, so even if a pod gets **rescheduled**, it still maintains the **same identity/name**.
- **Databases** are typically **stateful**, meaning they require **persistent storage** to store their data. They also require **stable network identities** to ensure that client applications can consistently connect to the same instance of the database
- If you deploy a database using a regular Deployment, Kubernetes will create multiple replicas of the database, each with its own randomly assigned hostname and IP address.

StatefulSet Characteristics

Predictable Pod Name:

- In a StatefulSet, each Pod is assigned a predictable name based on the name of the StatefulSet and its index.
- For example, if the StatefulSet is named "webapp" and has three replicas, the Pods will be named "webapp-0," "webapp-1," and "webapp-2." This allows for easy identification and reference to specific Pods within the Set.

Fixed Individual DNS Name:

- StatefulSets provide a fixed individual DNS name for each Pod, based on the predictable name assigned to it. This allows applications to refer to each Pod by a consistent DNS name, even if Pod is rescheduled to a different node.
- For example, if the StatefulSet is named "webapp," and the Pod is named "webapp-0," the DNS name for that Pod will be "webapp-0.webapp".

Headless Service:

- StatefulSets are accompanied by a headless service, which allows for direct communication with individual Pods rather than the Service as a whole.
- This is useful for stateful applications that require direct communication between Pods, such as database clusters.

Ordered Pod Creation:

- StatefulSets ensure that Pods are created in a specific order, with each Pod waiting for the previous one to be ready before starting.
- This is particularly important for stateful applications that require specific sequencing of events, such as database clusters.

StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-hs
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:latest
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "yourpassword"
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
```

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-hs
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
  clusterIP: None
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
  labels:
    type: local
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  nfs:
    path: /srv/nfs/kubedata/pv3
    server: 192.168.49.1
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: manual
  resources:
    requests:
      storage: 1Gi
```

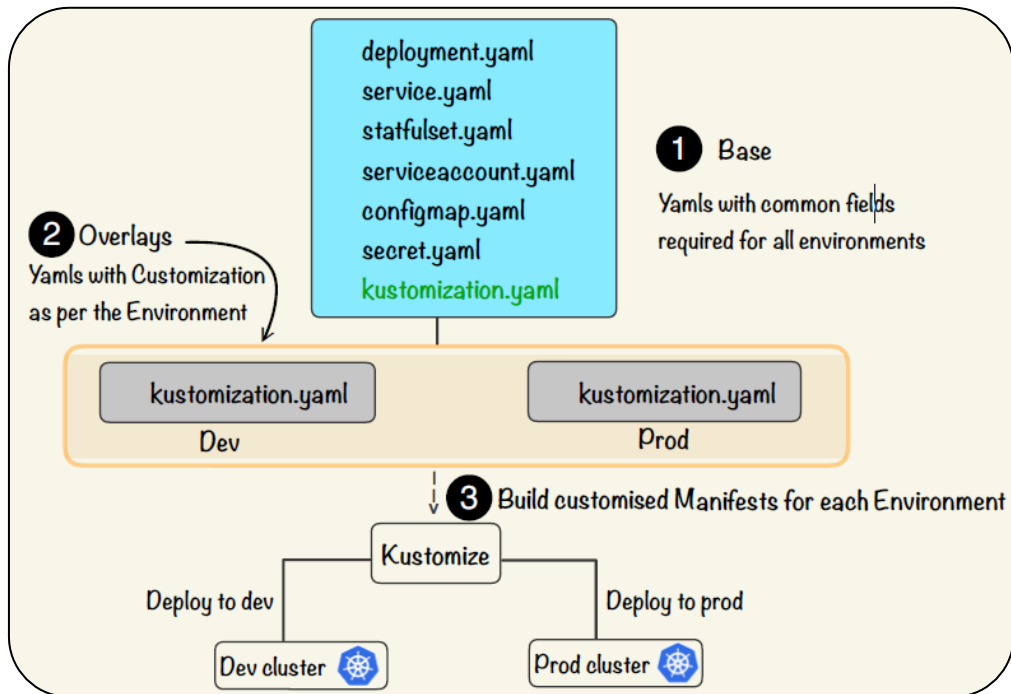
Deploy Applications on K8s

- An application in Kubernetes typically consists of multiple YAML files that define the k8s resources needed to run the application, such as Deployments, Services, ConfigMaps, and Secrets.
- You can deploy the application in Kubernetes manually by creating the YAML files and then using the `kubectl apply` command to create the Kubernetes resources on the cluster.
- Alternatively, you can use deployment tools like [Kustomize](#), [Helm](#), or the Helm [Operator](#) to automate the deployment process and simplify the creation of the YAML files.
- These tools provide a higher-level abstraction for managing Kubernetes resources and can make it easier to deploy and manage complex applications in Kubernetes.

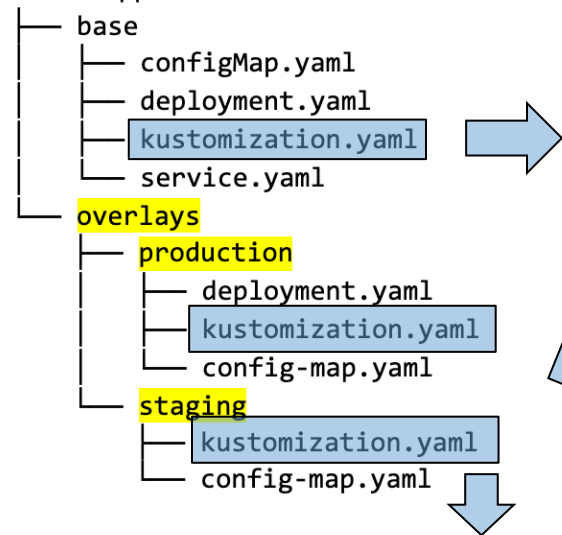
Kustomize

- **Kustomize** is a tool for managing k8s manifest files using a **declarative approach**.
- It allows to define a **set of base manifests** that define the desired state of your Kubernetes resources, and then **apply changes** using **composition** and **customization**.

Steps	Description
1	Create a base directory containing your Kubernetes manifests . This directory represents the desired state of your application or environment.
2	Define a kustomization.yaml file in the base directory. This file specifies the base resources to use, as well as any additional resources that should be added, modified, or removed.
3	Create overlay directories for each environment or application variant , if needed. These overlay directories contain additional resources or modifications to apply on top of the base resources.



~/demoApp



```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
- statfulset.yaml
- serviceaccount.yaml
- configmap.yaml
- secret.yaml
```

We create two overlay directories: STG and PROD.

Each overlay directory contains a kustomization.yaml file that specifies the **base** to use and any **patches** to apply.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
bases:
- ../base
namePrefix: stg-
patchesStrategicMerge:
- |-
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: nginx-deployment
  spec:
    replicas: 3
```

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
bases:
- ../base
namePrefix: prod-
patchesJson6902:
- target:
    kind: Deployment
    name: nginx-deployment
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 5
```

bases specifies the base directory to use, in this case **../base**

if the original name of the deployment resource is "nginx-deployment", it will be renamed to "prod-nginx-deployment".

The patch replaces the replicas field with a value of 5.

patchesJson6902 field is used to apply JSON patches, which are a more flexible and expressive way to modify resources compared to **patchesStrategicMerge**.

This kustomization.yaml file applies a patch to the **nginx-deployment** resource in the base directory, and adds a prefix "stg-" to the metadata name of all resources

Helm

- Helm is a **package manager** for **Kubernetes** that simplifies deployment of apps on a K8s cluster.
- It helps to **package applications** as **charts**, which are **reusable** and **shareable** bundles that contain all the resources required to deploy an application on a K8s cluster.
- Additionally, Helm supports **versioning**, which allows users to track changes to their applications over time and roll back to previous versions if necessary.
- Helm uses a packaging format called **"charts"**. A chart is a collection of files that describe a related set of Kubernetes resources. For example, instead of manually creating deployments, services, and other K8s objects, you can package these into a Helm chart.

Helm Files	Description
Chart.yaml	This is the core file which includes the name, description, and version of the chart. This file is used by Helm to identify the chart and to provide information to the user when installing or upgrading the chart
Values.yaml	This file contains the default values for the chart's parameters. These parameters are used in the templates to generate the Kubernetes YAML files. The user can override these values during installation or upgrade using the --set flag or a values file. This file is used to allow users to customize the behavior of the chart without modifying the templates directly.
Templates/	This directory contains the Kubernetes YAML files that define the resources to be deployed. These files are usually written in a templating language like Go templating or Helm's own template language. The templates can include placeholders for the values defined in the values.yaml file. The templates can also include logic to conditionally include or exclude resources based on the values of the parameters
Helpers.tpl	This file contains reusable snippets of code that can be used in the templates. These snippets can be used to simplify the templates and make them more readable. For example, a helper function might generate a random password or generate a unique name for a resource.

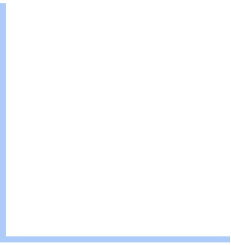
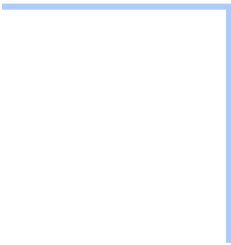
Helm Installation and its Usage

- How to install helm on a machine that has access to cluster:

```
$ curl https://baltocdn.com/helm/signing.asc | apt-key add -  
$ apt install apt-transport-https -y  
$ echo "deb https://baltocdn.com/helm/stable/debian/ all main" | tee /etc/apt/sources.list.d/helm-  
stable-debian.list  
$ apt update; apt install helm
```

- Add a Helm Repository from [Artifacthub](https://artifacthub.io) and install any applications that you want!

```
$ helm repo list  
$ helm repo add k8s-dashboard https://kubernetes.github.io/dashboard/  
$ helm repo update  
$ helm install my-k8s k8s-dashboard/kubernetes-dashboard  
$ kubectl proxy  
$ kubectl get pods  
$ helm list; helm uninstall my-release
```



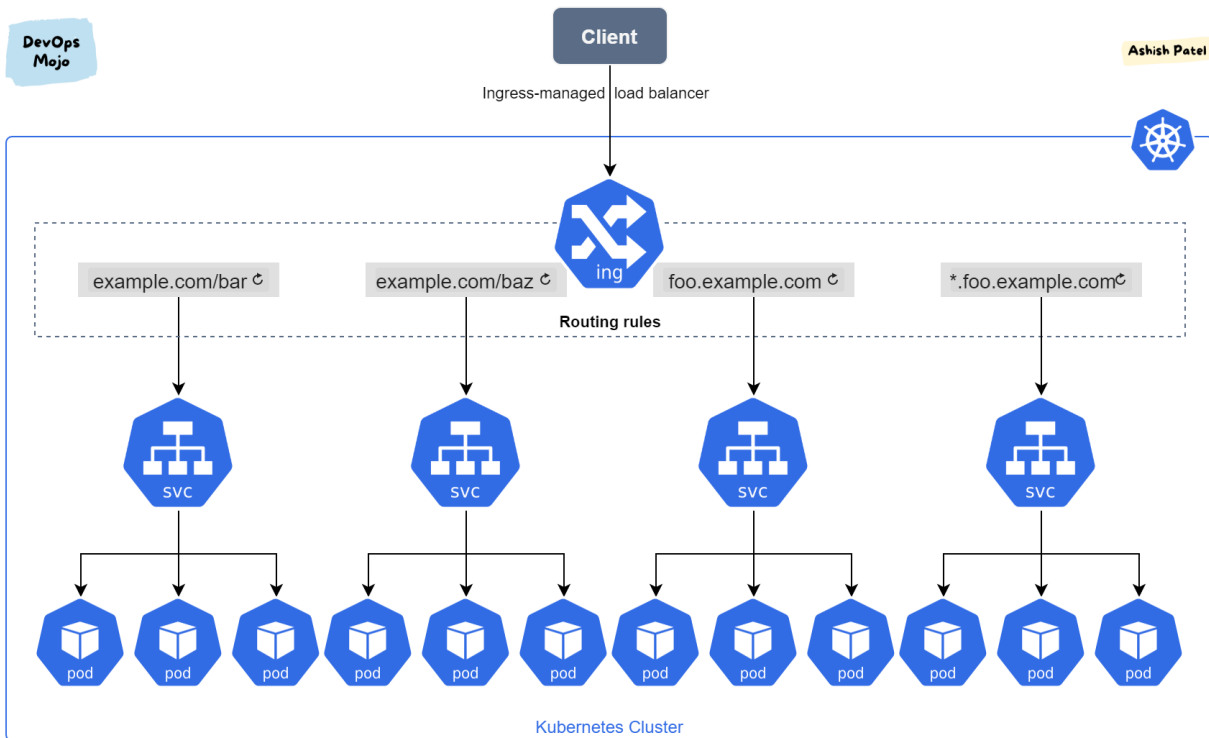
Ingress

Ingress

- **Ingress** an object that allows **access** to **services** from **outside** of the cluster.
- It provides **load balancing**, **SSL termination** and **DNS hosting** for your services.
- Ingress provides **layer 7** load balance routing to different SVCs like a **Rev-Proxy**.
- It allows to **expose multiple services** through a **single IP address**.

DevOps
Mojo

Ashish Patel



- **Ingress Resource:** is a K8s resource that defines the rules for how external traffic should be routed to services.
- **Ingress Controller:** is responsible for implementing the rules defined in the ingress resource and handling external traffic based on these rules. It acts like Nginx and use ConfigMaps to store configuration for the ingress resources and dynamically generate Nginx configs based on the rules defined in ingress resource.

Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-example
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /mail
        pathType: Prefix
        backend:
          service:
            name: mail-service
            port:
              name: http
      - path: /login
        pathType: Prefix
        backend:
          service:
            name: login-service
            port:
              number: 80
```

1

The ingress controller uses the **service name** specified in the **ingress rules** to lookup the IP addresses of the pods backing that service. It then routes traffic to those pods according to the path matching rules defined in the ingress.

```
$ kubectl describe ingress ingress-example
```

```
Name: ingress-example
Namespace: default
Address:
Default backend: <default>
Rules:
Host Path Backends
-----
example.com
/mail mail-service:80 (10.1.1.2:5678, 10.1.1.3:5678)
/login login-service:80 (10.1.1.4:5678, 10.1.1.5:5678)
Events: <none>
```

```
server {
  server_name example.com ;
  listen 80 ;
  listen 443 ssl http2 ;
  location /mail/ {
    set $namespace "default";
    set $ingress_name "ingress-example";
    set $service_name "mail-service";
    set $service_port "http";
    set $location_path "/mail";
  }
  location /login/ {
    set $namespace "default";
    set $ingress_name "ingress-example";
    set $service_name "login-service";
    set $service_port "http";
    set $location_path "/login";
  }
}
```

2

The Ingress Controller continuously watches for updates to Ingress Resources, Service, and Endpoints objects. When it detects a new or modified these objects, the controller is notified. it reads the information in these objects to understand what traffic routing changes it needs to make.

Ingress resources define rules for routing HTTP/HTTPS traffic to services in a cluster. They specify path-based rules that **map URLs to backend services**.

Enable Basic Authentication for an Ingress Rule

- An example for add **authentication** in a **Ingress rule** using a **secret** that contains a file generated with **htpasswd**.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example.com-admin
  namespace: default
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/auth-type: basic
    nginx.ingress.kubernetes.io/auth-secret: shop-basic-auth
    nginx.ingress.kubernetes.io/auth-realm: Authentication Required
spec:
  rules:
  - host: shop.com
    http:
      paths:
      - path: /admin
        pathType: Prefix
        backend:
          service:
            name: management-service
            port:
              name: http
```

```
$ htpasswd -nbm arash Foroughi | base64
$ kubectl create secret generic example-basic-auth --from-literal=auth=<base64 output>
```

or

```
apiVersion: v1
kind: Secret
metadata:
  name: example-basic-auth
  namespace: default
data:
  auth: YXJ5ZT0kYXBByMSQxbzAzWEltQjRJVfYudWh0dmcuVmV5d0t5a0s1cC4vCgo=
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-path
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
  - hosts:
    - example.com
    secretName: tls-secret
  rules:
  - host: example.com
    http:
      paths:
      - path: /opinions
        pathType: Prefix
        backend:
          service:
            name: opinion-service
            port:
              name: http
```

Enable SSL for an Ingress Rule

<https://kubernetes.github.io/ingress-nginx/examples/>

<https://github.com/kubernetes/ingress-nginx/blob/main/docs/user-guide/nginx-configuration/annotations.md>

METHOD-1 --> Self-Signed Certificate:

```
$ openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out tls.crt -subj "/CN=example.com"
```

```
$ kubectl create secret tls tls-secret --key tls.key --cert tls.crt
```

METHOD-2 --> User Certbot

```
$ certbot --manual --preferred-challenges dns certonly -d example.com
```

```
$ kubectl create secret tls tls-secret --key privkey.pem --cert cert.pem
```

Enable SSL for an Ingress Rule

METHOD-3 --> Use Cert-Manager

- **Cert-manager** is a **certificate management controller** for k8s. It helps with **issuing** and **renewing** certificates from various sources, such as [Let's Encrypt](#), [HashiCorp Vault](#), [Venafi](#).
- It ensures certificates are **valid** and **up to date**, and will attempt to renew certificates at a configured time **before expiry**.
- Cert-Manager in K8s will configure with two resources of **ClusterIssuer** & **Certificate**:
 - **ClusterIssuer**: is an object that represents a particular Certificate Authority or a specific method for issuing certificates. It defines the parameters and configurations required to request certificates.
 - **Certificate**: resource allows to specify the details of the certificate for request. They reference an issuer to define how they'll be issued.

Thank You!

Arash Foroughi

2023-24

SEMATEC COMPANY
